

TDK-thesis

Etele Jakab

Augmented Failure Report-Based Recommendation Systems in Manufacturing (Troubleshooting Ai)

EÖTVÖS LORÁND UNIVERSITY

FACULTY OF INFORMATICS

DEPT. OF ARTIFICIAL INTELLIGENCE



Author:

Etele Jakab

Computer Science MSc

II. year

Supervisors:

Natabara Gyöngyössy

PhD Student

László Horváth, Andor Hofecker

Robert Bosch Kft. Mentors

Budapest, 2025

Abstract

The high volume series manufacturing can have various predictable and known errors as well as unpredictable or unknown errors may become the cause of machine stops at production lines. During the examination of these situations, the machine operators briefly and freely formulate the error picture in their mother tongue according to the following structure: description, reason, and action. Currently, the database stores this information along with some additional data that may provide a better insight into the context of the errors occurrence.

The state of the art idea for retrieving information from free text entries is by the vectorization (generating contextual embeddings for the texts) and then by Euclidean distance-based vector searching for finding the nearest-neighbors. The answer is generated by paraphrasing the selected options by another Larger Large language model (LLM) like the Generative Pretrained Transformer (GPT) model.

The main issue of this concept is that the contextual differences between the raw entries on their own can be minimal or even misleading, given the fact that the same words can be used for different situations. A very simple but rather common example from our research are the entries of "Dropouts." or "Serial dropouts." which will have almost the same embedding but in the background the first one might be related to the process of **Screwing**, the second one might be related to the process of **Soldering**.

This research has multiple goals. The first is to investigate different augmentation methods and techniques for broadening the information content of the raw entries. The second goal is to investigate different entry similarity searching algorithms, given the fact that simple vector searching has some logical deficits. The third and final goal is to investigate different answer generation approaches, such as from how many possible entries should we forge the answer and what kind of prompts should we give to the verification model for evaluating a generated answer set.

Keywords: contextual and text augmentations, answer selection, answer verification

Acknowledgments

I would like to express my gratitude to all those who supported me in the preparation of this thesis. I would like to express my gratitude to my university supervisor, Natabara Gyöngyössi, who introduced me the field of Advanced Natural Language Processing and Deep Neural Networks, talked about this topic, encouraged me to work out this research and helped me with his advices along the process.

I would like to thank my mentors at Robert Bosch Kft., László Horváth, who introduced to me the field of Industrial Manufacturing and Engineering Problem Solving, and Andor Hofecker, who introduced to me the field of AI Engineering and the project of Troubleshooting AI. I am also grateful for the many important professional and scientific advices and for the immense amount of help that they have provided me throughout my work. Without that, this study would not have been completed. I would also like to thank the help of all of my other colleagues at Budapest-ME/ECO (Mobility Electronics - Engineering Common Competences) department.

Last but not least, I would like to thank my parents, Barna-Róbert Jakab and Irma-Tünde Jakab, who motivated and supported me throughout my scientific career. This thesis would not have been possible without the help of my teachers and family.

I am very grateful to all of them!

Contents

1	Introduction	3
1.1	The importance of continuous production	3
1.2	Existing workflow and possible improvements	4
2	Presentation of the Problem	7
2.1	The importance of line production continuity	7
2.2	Questions investigated during the research	8
2.3	Hypothesis	9
3	Complication Understanding Compass (CUC)	11
3.1	Introduction and Method analytics	11
3.2	Presentation of the developed method	16
3.3	CUC - Data Collection	23
4	Presentation of the data and its attributes	24
4.1	Data Attributes	24
4.2	Data Distribution	26
5	Presentation of developed End-to-End system	29
5.1	Tools and Technologies used for the development	29
5.2	Goals and limitations of the workflow design	31
5.3	Presentation of the workflow	33
6	Text Augmentation	37
6.1	Introductions	37
6.2	Incremental text augmentation	38
7	Text grouping	43
7.1	Introduction	43

7.2	Hierarchical clustering	44
7.2.1	Approach Description	44
7.2.2	Test Results	45
7.2.3	Final conclusions	48
7.3	KMeans clustering	49
7.3.1	Approach Description	49
7.3.2	Test Results	50
7.3.3	Final conclusions	53
7.4	DBSCAN clustering	53
7.4.1	Approach Description	53
7.4.2	Hyperparameter Search	54
7.4.3	Test Results	57
7.4.4	Final conclusions	58
8	Answer Selection Verification	59
8.1	Introduction	59
8.2	Evaluation metrics	61
9	Final Results	64
9.1	Numerical Results (Vectorial Searching)	64
9.2	Answer Verification Metrics	67
10	Summary of the Thesis	69
10.1	General results	69
10.2	Evaluation of the initial Hypothesis	70
10.3	Future developments	71
	Bibliography	74
	List of Figures	78
	List of Tables	80

Chapter 1

Introduction

1.1 The importance of continuous production

The automotive industry has long been a key player of the global economy, the manufacturing of vehicles being a complex and intricate process. One of the foundational elements of this industry is serial production, which involves the mass construction of vehicles on different assembly lines. These systems requires a high level of precision, efficiency, and coordination in order to meet the demands of the market and ensure the quality of the final product. However, serial production also presents a number of challenges. These include the need to constantly improve and innovate processes, manage complex supply chains, and meet strict quality and safety standards. In addition, the industry is also facing increasing pressure to reduce its environmental impact and adopt sustainable manufacturing practices.

In order to address these challenges and improve the efficiency and effectiveness of serial production, many automotive companies are turning to Data Science and AI software tools. These technologies can help to analyze and optimize production processes, identify areas for improvement, predict and prevent potential issues before they occur. They can also help to streamline supply chain management, improve quality control, and reduce waste and energy consumption. Furthermore, Data Science and AI tools can also play a crucial role in enabling the automotive industry to adapt to the growing trend of electric and autonomous vehicles. These technologies can help to develop and test new technologies, improve the performance and safety of vehicles and enhance the overall customer experience.

This study aims to support and to develop the production continuity aspect of this

field, given the fact that the worst thing that can happen at a production plant is the lack of production itself, including the chaoticness of production. The reasons for this we can divide this into two categories: economical and product quality/reliability.

The obvious element of the economical aspects is the fix cost of product manufacturing consist of amortization of investments (machines), the facility operational cost (like shopfloor heating, lightening) and the personal costs (e.g. white collar engineers and the maintenance, service blue collar staff). An example for this aspect is that in 2023 the Hatvan plant in Hungary generated a revenue of 692.686.620 thousand HUF [1]. In the automotive sector the production related costs are up to 80-85% of the revenue [2]. On the other hand in that very plant there are around 4900 workers[3] whose wages must be paid to ensure the production output.

The second aspect comes from the fact that in order to manufacture products professionally, ensuring their quality and reliability, the production steps and their ideal flow must be followed as precisely as possible, given that these are essentially a set of real-time physical processes. Based on this, it is crucial that production flows smoothly and predictably over time, otherwise the products may simply be scrapped on the line already. A particularly good example for this possibility is the **dispensing** or **gluing** process, given that if these steps are applied to a intermediate assembly phase, the clock is ticking so that the next operation has to be carried out in time.

1.2 Existing workflow and possible improvements

The ideal state of manufacturing is continuous production without any disturbance from type changes and without any disturbance from planned stops (e.g. maintenance) or non-planned stops (e.g. machine failures, high amount of product quality failures). The following workflow introduce the case of non-planned production stop due to machine or process failures:

- Step 0: A machine operator is called for service with a failure message on the HMI (Human machine interface), ideally with a failure category and code.
- Step 1: A machine operator is called for service with a failure category
- Step 2: If a machine operator cannot handle the failure, than calls for a higher hierarchy support, which is a service technician. A service technician accepts the tasks.

- Step 3: The service technician gets the problem overview of the machine
- Step 4: The service technician uses his own guidance knowledge or that available in current systems for action planing
- Step 5: The service technician, in case it is necessary, collects additional information sources, documentation, and instructions.
- Step 6: The service technician does actions (tries different problem solving approaches) until the problem is fixed
- Step 7: The service technician documents the problem and the actions taken
- Step Final: The issue is closed, the event experiences are saved to the database and the production is restarted.

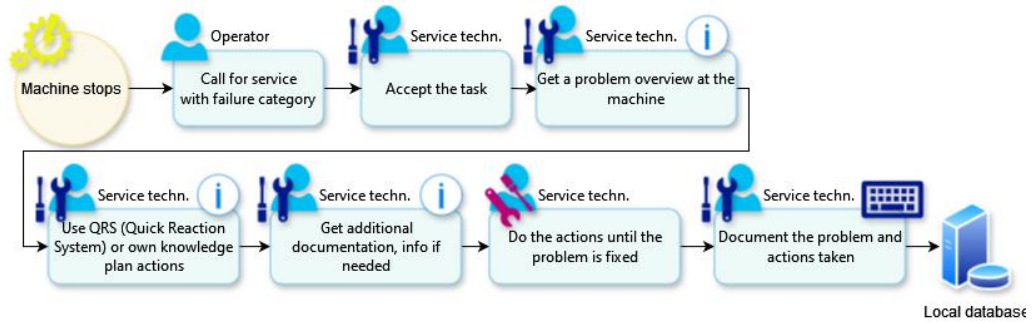


Figure 1.1: Current Problem Solving Workflow

When examining this workflow, a serious vulnerability clearly emerges. Namely that information gathering and action planning are done entirely at the human level. This means that, for example, a new or inexperienced colleague is essentially blind to where start looking for useful instructions. Moreover, even more experienced employees can experience "brain drain" when faced with a rare error, given that not even they have enough or any experience at all with such situations. Therefore, we suggested an (Figure 1.2) improved workflow with the integration of the Generative Ai steps: First, we replaced steps 4 and 5 with the following new actions:

- Step 4(new): The service technician uses the platform to input the problem description
- Step 5(new): The AI chat-bot does its processes and generates an answer with the best possible actions to be taken

- Step 6(new): The service technician selects an action to take

And for second, between the original step 6. and step 7. we have integrated the following new step:

- Step 9(new): The service technician gives feedback to the AI about the generated possibilities and or about the whole answer

GenAI improved workflow

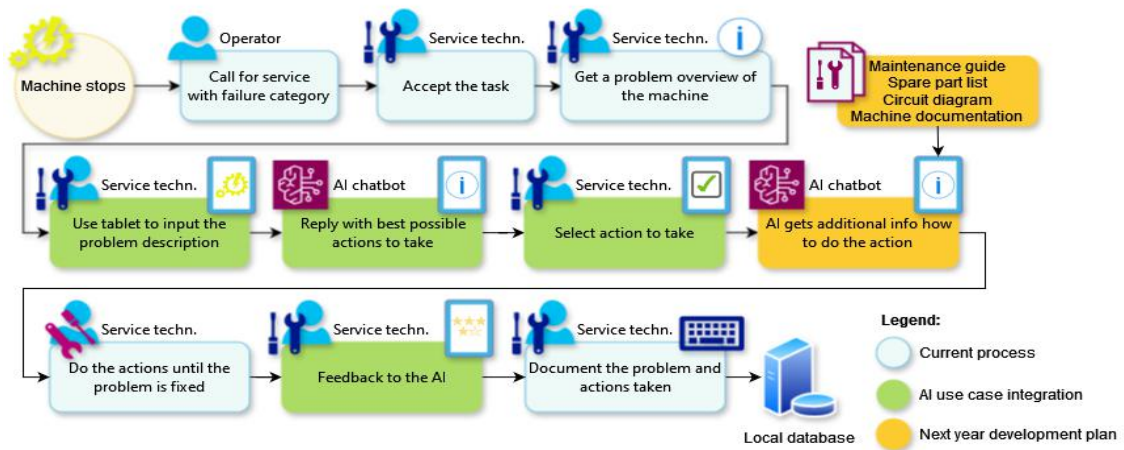


Figure 1.2: Improved workflow, green boxes representing the newly integrated steps

These shown advancements in the process intend to give immediate guidelines to the staff which shortens the necessary time for determining the reason and supposed actions for solving the problem. Here we can not only use the potential of the direct Pareto principle, but also the inverse of it, namely that there are failure types that are more rare and in general takes way more time to be detected. An another potential benefit of AI chat-bot application in case if there is a fluctuation of staff employees due to changes in persons, functions or organizations. Furthermore, if a machine or a line itself relocated into a new organizational unit with new employees, the chat-bot still can support without language or knowledge barriers. In case of new colleagues, it is a great help to have immediate suggestions for handling the arisen issues which allows for shorter problem solving time and that is an ultimate aspect of the general production efficiency.

Chapter 2

Presentation of the Problem

2.1 The importance of line production continuity

Considering that our starting point is in size a large dataset, containing low quality and information poor entries in Hungarian, first we have to get to the point where we can already handle them with large language models trained on English documents. There are two reasons for translating this content. The first one is a rather technical one, considering that the best embedding models are trained on English texts. These models are not just more performative, like having more parameters and or trained for longer time, but the training data that they get in connection with is larger and covers way more text compositions and therefore can generate a way more accurate contextual representations about our entries.

Apart from the technical perspective, we also have to take that into consideration, that Robert Bosch Kft. has plant across the globe, more specifically 22 manufacturing locations in 13 different countries and therefore different languages in. This is can be helpful from one side, because we ultimately have a huge amount of information about problem solving processes in manufacturing, but that comes from the named amount of sources. Therefore, it is not only logical to translate all that to a common language, but thanks to this, there will be no need to develop a new system for every single language for later steps. We can use a standard one which will receive the most amount of focus and development resource for all of its sub-parts. This is an ideal approach from both a business and software improvement perspective.

Based on these, the steps of this research project are as following:

1. Heuristic textual preprocessing

2. Translation to English
3. Data Augmentation
4. Contextual Embedding
5. Entry grouping and Answer Selection
6. Answer Verification

Considering the abstract technical scope of our investigation, we will mainly focus on the experimentation and development of points 3, 4 and 6.

2.2 Questions investigated during the research

In this section, we will raise and explore the key questions examined during our research, which are designed to deepen our understanding of the challenges and opportunities within the field of text generative AI. By addressing these questions, we aim to uncover insights that will guide our development process and enhance the effectiveness of our application. Each question has been carefully formulated to target specific aspects of our research objectives.

1. How can filter out entries that by default, do not contain relevant information?
2. What kind of simple data preprocessing steps are necessary for even beginning the investigations of the data?
3. What kind of tokens-words are present in the entries?
4. What is the internal distribution of the data? (machine, process type etc.)
5. What are the attributes of the benchmark embeddings?
6. What are the most important information packages that we should augment with?
7. What is the order of importance for information's?
8. What is the proper way for augmentation? On text level or vector level?
9. What is the best searching method of the entry grouping?
10. What is right amount of options from which the suggestions should be generated?
11. How should we formulate the answer verification prompts?

2.3 Hypothesis

As shown in previous paragraphs, this work contains multiple research areas therefore multiple hypothesis statements can be set, not only on individual field level, but even within each topic or by the combinations of them. By summarizing the questions described before, the following hypotheses are formulated aiming to show the assumptions investigated in this thesis.

1. Augmentation:

HA0: Attaching contextual information on text level to the raw entries will diverge those that has high similarity at base form.

HA1: Attaching contextual information by vectorial operation to the raw entries will diverge those that has high similarity at base form.

HA2: Too much contextual information will have noise effect on the entries, meaning, that the resulted groups will resemble more the contextual similarities between the added augmentations.

HA3: Text level augmentation (Early fusion) will perform better than the separate vectorization and vector level operation augmentation (Late fusion) of the additional data.

2. Entry grouping and Answer selection:

HEg0: Hierarchical clustering will have a better grouping effect of similar instances than direct vectorial searching of closest neighbors.

HEg1: KMeans clustering will have a better grouping effect of similar instances than direct vectorial searching of closest neighbors.

HEg2: DBSCAN algorithm will have a better grouping effect of similar instances than direct vectorial searching of closest neighbors.

HEg3: KMeans clustering will perform the best out of the investigated clustering algorithms.

3. Answer Verification:

HAV0: Within the 10 best matches of the answer possibilities by numerical attributes will contain at least one right answer.

HAV2: Within the 10 best matches more than half will be an acceptable answer option.

HAV3: A simple output prompt (like returning just a number) for verifying the answer set, will be able to provide trustworthy results.

Chapter 3

Complication Understanding Compass (CUC)

3.1 Introduction and Method analytics

In the dynamic landscape of industrial and serial production environments, effective problem-solving is crucial for maintaining operational efficiency and driving continuous improvement. The inherent complexities of these settings demand a systematic approach that not only addresses immediate challenges but also fosters long-term enhancements. By employing structured methodologies, organizations can identify inefficiencies, streamline processes and reduce waste ultimately leading to higher productivity and quality. This systematic approach ensures that solutions are not only effective but also consistent, allowing teams to replicate successes across various production lines.

Furthermore, fostering a culture of continuous improvement empowers employees to contribute their insights and innovations, creating a more agile and responsive production environment. As we delve deeper into the intricacies of problem-solving in these contexts, it becomes clear that a methodical framework is essential for achieving sustainable growth and operational excellence.

In the context of this research, the goal was to create a framework for collecting relevant information packages for the purpose of the contextual augmentation. The very abstract idea was, that we should consider the situation as if we were in the middle of the ocean (indeed we face the infinite sea of information) and we need a guiding device, such as a compass, for exploring the right islands of information (sole or combination of multiple instructions) that we can jump on for mapping out the entire situation. For this

reason, it was necessary to investigate different well established problem solving methods and choose the appropriate or the combination of the appropriate methods that can present the basis of the augmentation-al operation.

In the following we would like to present all the approaches that were the subjects of our development:

1. **Six Sigma:** In industrial and serial production environments, the Six Sigma problem-solving method is a powerful framework for enhancing quality and efficiency [4]. This data-driven approach focuses on eliminating defects in processes, minimizing variability and ensuring consistent output. The Six Sigma methodology uses DMAIC—Define, Measure, Analyze, Improve, and Control—to guide teams through a systematic investigation. The Define phase outlines the problem and sets objectives, while the Measure phase collects data to assess performance levels, including production rates and defect rates. In the Analyze phase, teams identify root causes using statistical tools. The Improve phase develops solutions, and the Control phase ensures improvements are sustained through monitoring. By following this structured approach, organizations can resolve challenges and foster a culture of continuous improvement, driving operational excellence [5].

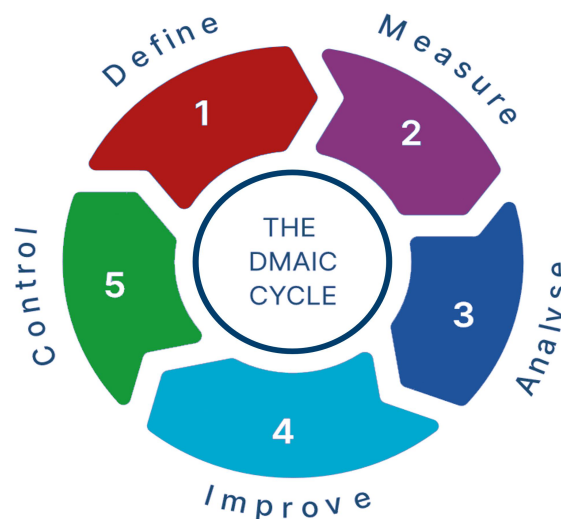


Figure 3.1: Six Sigma

2. **Shainin Red X:** In the fast-paced world of manufacturing and mass production, the Shainin Red X problem-solving method provides an effective approach to resolving complex issues [6]. This methodology focuses on isolating the "Red X," or primary cause of a problem, through a systematic, data-driven investigation. The process be-

gins with collecting information, including process data, defect rates, and operator insights, to understand the issue's scope. The Shainin approach involves key steps: defining the problem, conducting experiments to gather data to pinpoint the Red X, and analyzing this data to identify the root cause [7]. Unlike traditional methods that rely on statistical analysis, the Shainin Red X method encourages practical experimentation, allowing teams to validate hypotheses quickly. Once the Red X is identified, targeted solutions can be implemented to eliminate the root cause, improving quality and efficiency. The method also emphasizes sustaining improvements through ongoing monitoring and feedback. By leveraging the Shainin Red X approach, organizations can enhance their production processes and outcomes.

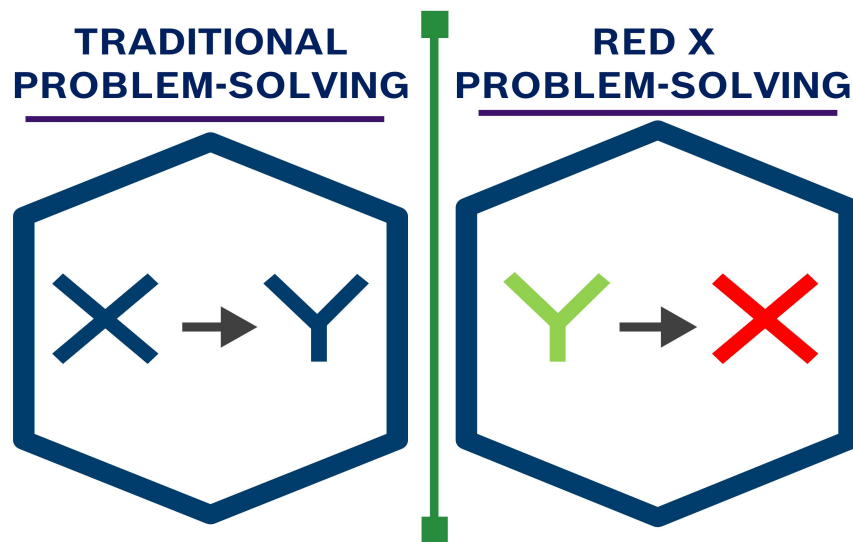


Figure 3.2: Shainin Red X

3. **Lean-CIP:** In the competitive realm of industrial production, the Lean methodology and Continuous Improvement Process (CIP) provide a framework for enhancing operational efficiency and quality [8]. Lean focuses on eliminating waste and optimizing processes ??, while CIP emphasizes ongoing enhancements through iterative problem-solving. The process begins by identifying value from the customer's perspective and collecting critical information such as process flow data, cycle times, defect rates, and employee feedback. This data is vital for mapping workflows and identifying inefficiencies. The Lean and CIP structure involves key steps: defining the problem, analyzing the current state, brainstorming solutions, implementing changes, and measuring results. Tools like value stream mapping and root cause analysis help visualize processes and identify bottlenecks. After improvements, the focus shifts to sustaining changes through regular monitoring and feedback loops.

By fostering a culture of continuous improvement, organizations empower employees to contribute insights, leading to innovative solutions and a more agile production environment. Ultimately, Lean and CIP drive immediate improvements and cultivate a mindset of excellence for long-term success [9].

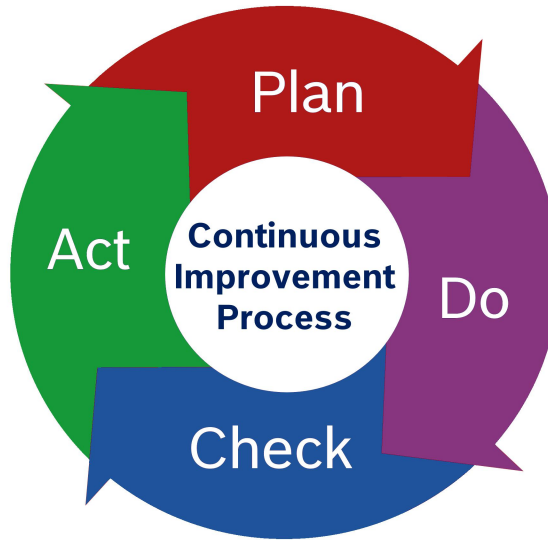


Figure 3.3: Lean-CIP

4. **Ishikawa:** In industrial and batch production environments, the Ishikawa problem-solving method, or Fishbone diagram, offers a structured approach to identifying root causes of quality issues [10]. This method helps teams visualize factors contributing to a problem, categorizing them into areas such as materials, methods, machines, measurements, environment, and people. The process starts with defining the problem and collecting relevant information, including defect rates and employee insights. Engaging cross-functional teams enriches the analysis. The Ishikawa method involves brainstorming potential causes and organizing them into the Fishbone diagram [11], identifying likely contributors. Once root causes are identified, targeted solutions can be developed and implemented. The effectiveness of these solutions is monitored through ongoing data collection, ensuring sustained improvements. By using the Ishikawa method, organizations resolve challenges and foster a culture of collaboration and continuous improvement, enhancing production processes and product quality.

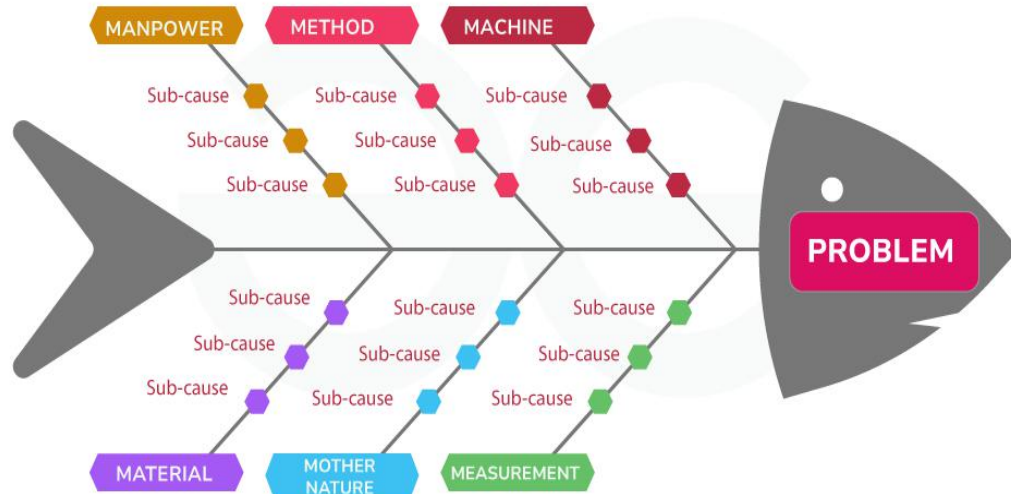


Figure 3.4: Ishikawa-Fishbone diagram

5. **Mind-map:** The Mind-map problem-solving method is a creative approach used in industrial production to facilitate brainstorming and organize complex information [12]. This technique allows teams to visually represent ideas and relationships, making it easier to identify issues and explore solutions. The process starts with a central problem at the center of the mind map, with branches extending to represent aspects like causes, effects, and potential solutions. Information collected may include production metrics, employee feedback, and equipment performance, contributing to a comprehensive understanding of the problem. As teams develop the mind map, they categorize and prioritize information, clarifying relationships and identifying root causes. This structure encourages collaborative thinking and exploration of multiple perspectives, fostering innovation. Once complete, teams can focus on critical areas, developing targeted action plans. By utilizing the Mind-map method, organizations enhance communication, streamline investigations, and drive continuous improvement in production environments [13]. This visual tool aids in problem identification and promotes a culture of teamwork and creativity.

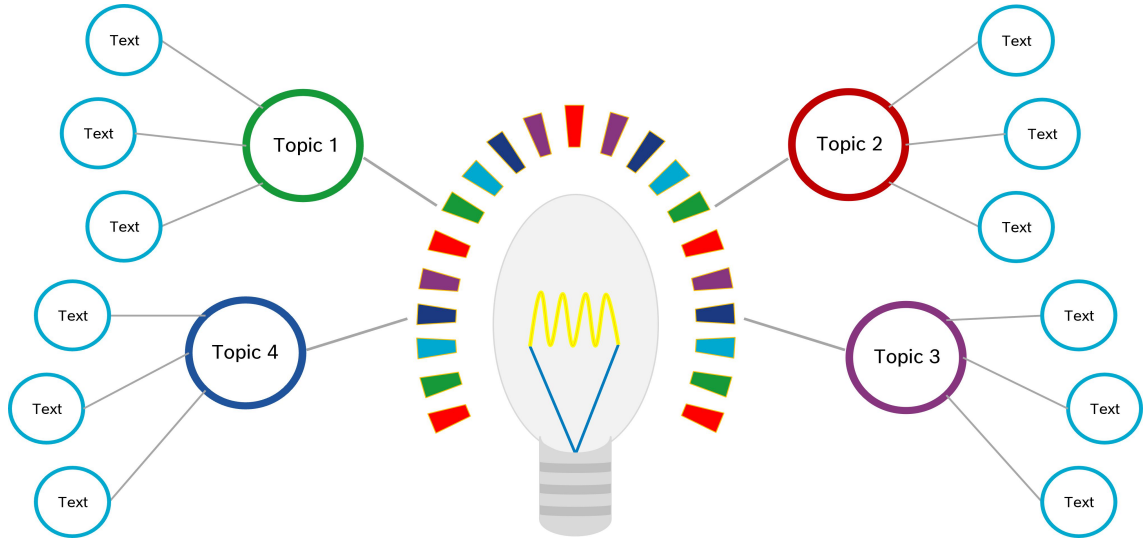


Figure 3.5: The Mind Map

3.2 Presentation of the developed method

In this research, we found the combination of two methods necessary to interlink the LLM with applied technical terminus-es and targeted procedures. The reason for this, was to focus on the contextual in relation to the specific problem and to the specific manufacturing environment. The selected methods are: the Ishikawa method, hence potential failure causes can be generated as contextual information and the Mind-map which gives the structural relationship and enhance the content of free text service logs by a usage of data bases containing information packages that are sorted in an order of importance.

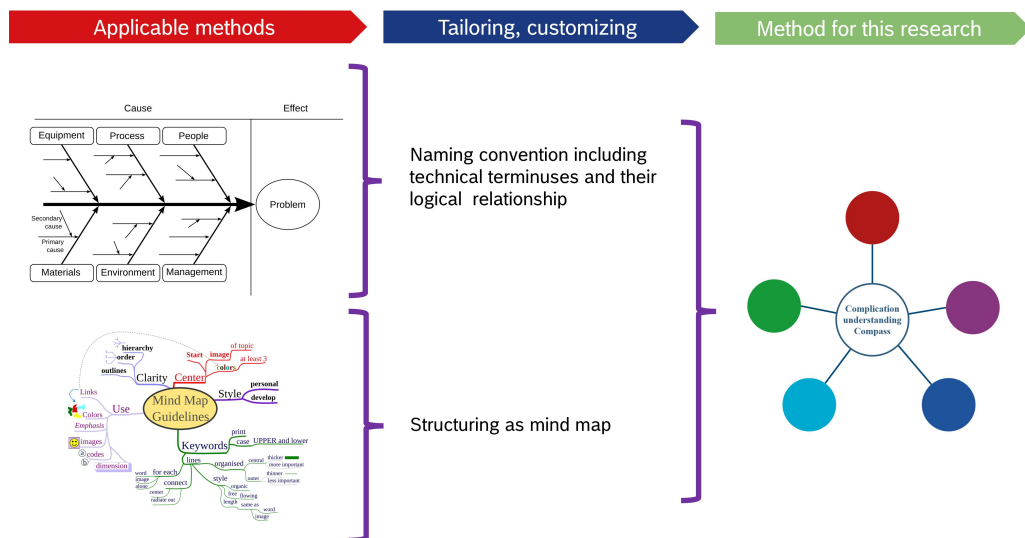


Figure 3.6: Framework Evolution

The potential causes of problems are divided into five primary factors: Machine, Material, Man-Maintenance, Method and Environment. These categories help teams systematically analyze and pinpoint the root causes of a production issue.

1. **Machine** is typically considered the most critical factor since production heavily relies on the functioning and type of machinery. The concept of 'Machine' is a transformation which includes input(s), processing(s) and output(s). Ensuring machines operate optimally reduces downtime and defects, significantly impacting overall productivity. This grand category is further divided into subcategories detailing the most important aspects of its operation:
 - (a) **Process**: The manufacturing process refers to the series of steps and operations involved in transforming raw materials into finished products. This process encompasses a variety of activities, including design, production, assembly, quality control and packaging. The goal of the manufacturing process is to create goods that meet specific standards and requirements efficiently and cost-effectively. Examples for this category are Screwing, Automatic Dispensing or Selective Soldering Multi-wave.
 - (b) **Process core**: These are the primary activities involved in manufacturing, such as machining, assembly, welding, molding, and other techniques, technologies that directly affect the product's form and function. The process core is where value is added to raw materials or incoming components or sub-assemblies. Examples for this category are Screwdriver, Tightening or Dispensing.
 - (c) **Positioning**: The components assembled during the manufacturing processes requires an ensured relative positioning. The positioning can be ensured by the product design, by manufacturing toolings or combinations during the manufacturing sub-assembly steps, like proximity, pre-guidance to position, guide in position, assemble/move in position operations, or stages within the overall manufacturing workflow. This concept is important for optimizing efficiency, minimizing waste, and ensuring smooth transitions between different phases of production. The positioning can be realized by different methods, dependent on the fixed and the moving elements during the process steps. Typically the large weight component is fixed and small weighted components are the moving elements. The movement can be realized by human operator hands, or

by automated movements, like CAM guided mechanisms, servo axis (X-Y-Z) systems or multi-axis (multi-degree) robot arms. The main differentiating factor is whether the assembling machine moves towards the product or inverse.

- (d) **Material feeding:** The materials or components can be fed into production processes via multiple methods, dependent on the function, fit and form of the component and the form of packaging. The most common options are the Work piece carrier, Plate or in Bulk
- (e) **Mode of operation:** There are 3 basic concepts available to describe the automation level of a machine: automated, semi-automated and manual. These terms can be used either for the previously described process types, but now within a station for transport, value addition and testing. The main material flow, and also incoming materials which are added toward the main material (or subassembly flow), can be automated with transport systems. Palletized products are waving from one step to the other. Transport can be semi-automated, where the weight supported by pallets, but the movement supported by human power. Or finally, it can be manually grabbed, picked and place by human operator. The other ways to move products within machines if there is a round-table or a drawer system applied, in order to separate the loading, unloading of the main material flow from the value addition process steps.

2. The next is **Material**, examining the raw materials or components used in production. This part of the Ishikawa diagram describe the potential failure cause from related variances or non-conformities. In practice, the main components are described here with their material usage, in daily language/wording like "aluminium shaft", "cover plastic" etc. Moving on to the subcategories, let's delve into the most important aspects of operation within this grand category:

- (a) **Component technology:** The electronic control units have typically components from materials like Cover or Housing (from aluminum die cast, aluminum sheet, plastic injection molded materials), PCB (multilayer), Sealing materials (typically 1 or component dispensed silicone materials) or TIM - thermal interface material (typically 1 or 2 component dispensed composite materials contain high viscous gap filler liquids embedded with heat conduc-

tive particles). Potting, gelling, coating materials are for component stabilization or avoidance of humidity in certain product areas.

- (b) **Package:** The most simple material packaging and feeding form is a bulk packed material. For fluids is in the form of syringe, cartridge or hobcock bucket. For solid materials typically in bags or boxes. The components which are sensitive to fit/form or aesthetic parameters, are typically fed to production machines in trays where the position of components ensured to avoid damage and enable removal methods in either manual or automated manufacturing process.
- (c) **Specific raw material (material groups):** The materials, in general, from assembly point of view can be the incoming sub-assembly as the main input through the primary material flow (e.g. subassembly on a workpiece or workpiece on a transport conveyor) and the additive materials which are assembled onto the incoming subassembly:
 - i. Quantifiable materials, e.g. screws, covers and etc.
 - ii. Consumable materials, e.g. dispensable fluids, meltable solid/liquid materials and etc.

The materials also can be detailed by the physical or chemical material properties which is a more conventionally used view: a part of a product which made from metal (like aluminum, steel) or plastics.

- (d) **Mechanical surface:** The mechanical surface describes the potential failures that can cause defect mainly during manufacturing processes. The mechanical surface includes the local geometrical conformity of the part, like roundness, concentricity and also the micro surface like roughness, burrs or hardness.
- (e) **Chemical surface:** The chemical surface describes the presence of substances in liquid or solid form over the mechanical surface, typically important for chemical bonding. There are materials which preferred to improve the technological function (e.g. adhesion of a sealant material to mechanical surface) or risking, causing failures like greasing or oiling, ultimately preventing adhesion of holding glues to mechanical surfaces. Example for this part are like wettability or any b) presence of substance (soap, oil etc.).

3. **Man-Management**, The man and management factor is key for efficiency and utilization of the manufacturing operation, which includes human factors and maintenance practices. This factor looks at operator skill levels, trainings and maintenance schedules. Turning our attention to the subcategories, let's examine the most important aspects:

- (a) **Operator function:** Operator function is dependent on the organizational structure of the manufacturing company. The key is the trust, control level and culture of the corporation. Ideally the employees are trained for their operational function and the decisions are made on the lowest possible level according to guidelines and instructions. The typical operational hierarchy begins with base trained operators, who perform low value adding tasks according to their direct work instructions. The next level is a machine operator, machine or line setter who understand the key operation of machines and able to switch production type and handle basic problems. The service, technician or engineer level members are able to solve complex problems in case of a breakdown. They are also able to define and improve process performance like cycle time or overall equipment efficiency including utilization rate.
- (b) **Maintenance Method:** Typically the engineering has the responsibility to define preventive maintenance, however, these are generating production time losses. It is still a state of the art question if a company should go with preventive maintenance, react only to failures or be able to apply predictive maintenance with monitoring and analysis systems applied within the production systems and machines.
- (c) **Organization and Structure:** The human resources-responsibilities in general are structured in a vertical or often in a matrix organization, which means that service employees that can be responsible for one process-machine type through the complete factory, through multiple departments or service as a pool. They can be responsible for a larger production areas with multiple production lines and large variety of machines and processes. And finally, service and-or maintenance can be anchored to a local area, such as single production line.

4. **Method** is the fourth factor, focusing on the processes and procedures employed in production. As methodical point of view, we should distinguish from the process,

pointing toward more the service interactions. The machines are running mainly in automatic mode, where a machine warning or error can be generated during continuous operation or can be consequence of previous service intervention actions. For example, typical case would be if something has to be frequently cleaned and calibrated, than the measured time or the counted processed part would be an important methodical parameter until the next warning or failure would occur. Now, let's break down this category to the most important aspects of its operation.

- (a) **First part after auto mode:** The appearance of machine errors or quality errors into the production systems often driven by the first part after switching to automatic mode, in this case the machine or tool status has changed.
 - (b) **First part after new material batch:** During automatic or continuous production, if a machine stop happens after the first part from a new material (another supplier, another production batch) loaded to the production machine, it is most probably the material change that caused the error. Here part number, supplier, and if available supplier traceability can reveal causal relationship.
5. Lastly, the **Environment** summarize factors which lead to failures (warnings, errors) in production. They can be externalities (temperature, time, humidity) and internalities, focusing on human factors (day or night shift, elapsed time since last break...etc.) Let's turn to the subcategories and examine the key variables that are most relevant within this extensive category.
- (a) **Shift time:** The time period of a calendar day which can influence the human error factor. For example during the night, monotonicity might not as tolerated like during the day which might lead to failures. However, further correlation studies required to strengthen this hypothesis.
 - (b) **Time after short break:** The focus could be more active if breaks are made during a continuous operation, which potentially improve the quality of work (even in a standardized work structure), therefore the time after break could be a potential correlation toward human related failure potential increase.
 - (c) **Temperature:** This can be understood in three different levels. One is the external temperature, which is weather dependent, the second is the production halls internal temperature, which has in many cases defined minimum and maximum temperatures, and the third level is the local temperature, which is

measurable within the machine internal areas, in product locations or within machine areas.

- (d) **Humidity:** Humidity and Temperature are heavily interconnected. However, it is worth treating them independently, considering that they exist as factors of different weights for the processes and tendencies that constitute the content of the research universe.

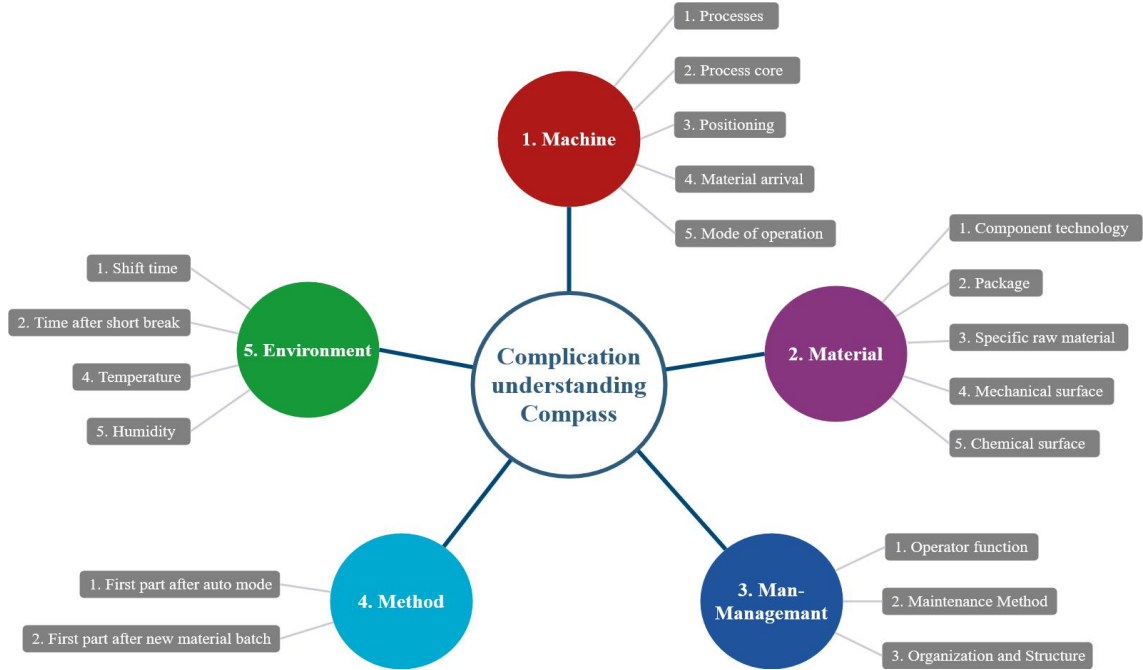


Figure 3.7: Complication Understanding Compass

By prioritizing these factors in this order, this method provides a structured approach for diagnosing and solving problems in industrial and serial production environments. Each category and its subcategories are examined in detail to ensure all potential causes are considered, leading to more effective problem resolution and process improvements or even unforeseen correlation detections between error types and certain contextual factors.

The main design viewpoint behind the presented scheme was generality. The reason for this is, that in automotive industry there are countless different production lines and value streams that have unpredictable digitization or information retrieval levels. Meaning, that even if we know, what kind of information are our preferences, they are often unavailable, or in practice even worse, semi available, given the fact, that the system can not have a general consistency level. Following this compass, at least we will always have a general prioritization order for any value stream in the whole of the industry.

3.3 CUC - Data Collection

The main realizational challenge of the whole project was building up necessary database. This was more or less and inherited consequence of the current digitalization level of the industry and the availability of these information. Technically, all necessary information's were available, but in unstructured and unfiltered format, like in pdf work-books or summary excels. This was a great data engineering work-package, which was further divided into three sub tasks: data collection, data preparation and data integration.

The first one was essentially a huge amount of manual labor. It was necessary to collect all the necessary documents related to the production lines and machines that participated in the project, and then literally searching for paragraphs or even sentences that could provide the target information.

Data preparation was still more of a monotonic but rather important task, the specific wording of the knowledge base. There were two main rules for this part: it had to be in English, given the fact that later we had to combine with the translated version of the raw entries, and they had to be as short as possible without containing specific specific only internally meaningful key words. About the reasons of this, there will be broader explanation later.

The data integration step is basically the research scope itself of this thesis. Figuring out the best way of merging the augmentational texts and raw entries was the hardest question between all the presented in hypothesis section. Should it be simple a concatenation of text? If so, should we include connecting parts? What should be the right order of concatenation? How many of contextual informational packages are enough or even too much for the improvement of the systems? This last aspect has proven to be particularly interesting.

Chapter 4

Presentation of the data and its attributes

4.1 Data Attributes

In this chapter we would like to present the most relevant attributes of the dataset that were the basis of this research. There are two main aspects for this investigation: what content conditions exist in the data and what qualitative properties characterize the set. The first perspective investigates the distribution of the theoretical failure categories, the distribution of machines where the errors occurred, the distribution of the entry authors or even temporal aspects like, how much time did it take to resolve the issues and distribution of error occurrences regarding the time of the day.

This work is based on the cases of production line machine malfunctions where there was a service technician involved. The data registration system of plants, generally speaking, automatically records every machine stop event that just happens, but if there is no need technical support, than no text data will be generated that could later be processed by a language model. Therefore, this research deals only with the fraction of the entire database, that contains the proper information.

Now lets present the actual source data of this research. First, we would like to present the structure of the dataset. By default (directly accessible), the data has 13 data-columns (features) and 11584 rows (failure cases). The features of the dataset are the following:

1. **production_line**: The identifier code of the production line where the error was registered.

2. **workplace**: The identifier code of the operational unit (machine) within production line where the error was registered.
3. **error_start**: The automatic registration time of the error.
4. **login**: The log in time of service technician into the data registration system, the human reaction time label. For not all cases of malfunctions are there log ins.
5. **reaction_time**: The elapsed time between the automatic failure registration and the human reaction (log in to the data registration system)
6. **service**: The reference information to the service technician who handled the situation
7. **logout**: The log out time of service technician from the data registration system. For not all cases are log outs, sometimes they are forgotten or the problem ended up unsolved.
8. **repair_time**: The elapsed time between the services technicians log in and log out from the data registration system.
9. **error_end**: The time label for the problem solving processes end. In the overwhelming majority of the cases this is equal with the "logout" but sometimes there are delays between the two.
10. **error_time**: The elapsed time between the automatic failure registration and the finalization of the problem solving ("error_start" and "error_end")
11. **error_code**: The short error code of registered malfunction. During data registration, this is chosen from a predefined list.
12. **error_cause**: The description of the cause for the machine stop. This features content varies in length, structure and quality as well since it is the freely formulated by the problem solving service technician in Hungarian.
13. **action**: The description of the act of problem solving that resolved the machine malfunction. This features content also varies in length, structure and quality and it is also the result of freely formulation by the problem solving service technician in Hungarian.

4.2 Data Distribution

Second of all, we have investigated the content conditions of the data. We have 3 main categorical divisors for our dataset. The first two, that are preliminary factors, are the "production_line" and the "workplace". They already present some level of contextual information for the registered cases, but in their raw format, they can not be further processed (the reason for this will later be presented). The third divisor, which is a subsequent information), is the "error_code" (the reason for this will also be presented later).

The quantitative distribution of the individual categorical labels is unbalanced, given the fact that in real life production some machines can malfunction more easily and some processes are more unstable. In case of the "production_line" there were 10 categories and for "workplace" there were 16 categories. After combining the "production_line" and "workplace" information, we got 19 cases and it is already clear from this that we don't have a complete combination between the two sets. Now, we would like to present the count distribution of these features normalized by the entire dataset size.

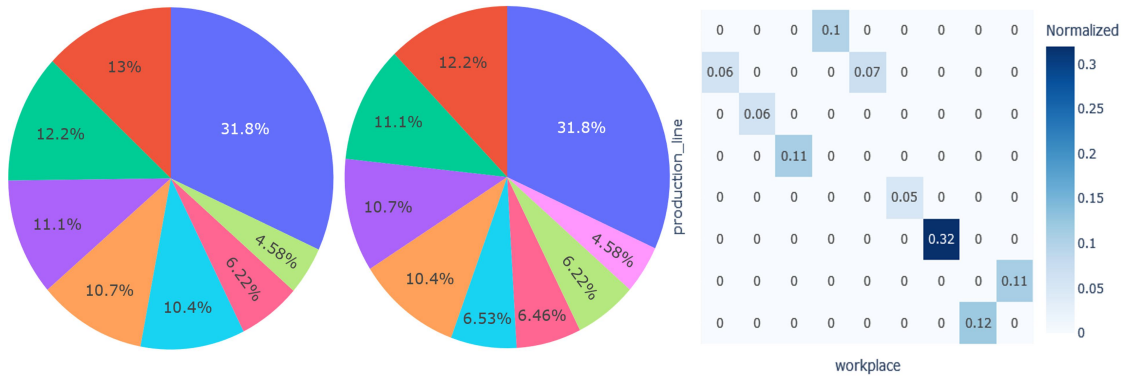


Figure 4.1: Distribution of production_line and workplace and their combination

As mentioned before, the data has the "error_code" feature, which contains short texts from a preexisting set. This is considered to be a subsequent categorical factor, because these are registered after the error occurrence and they essentially present a finite set of labels, which on their own contain information that would be a good starting point, but due to their nature, they do not contain the nuanced details and they are too superficial. For example they contain special code words and error ids. On the other hand, they can be used for cross checking and statistical weighting in the analysis of freely formulated text entries. In this case, there were 463 different values but the distribution was extremely imbalanced.

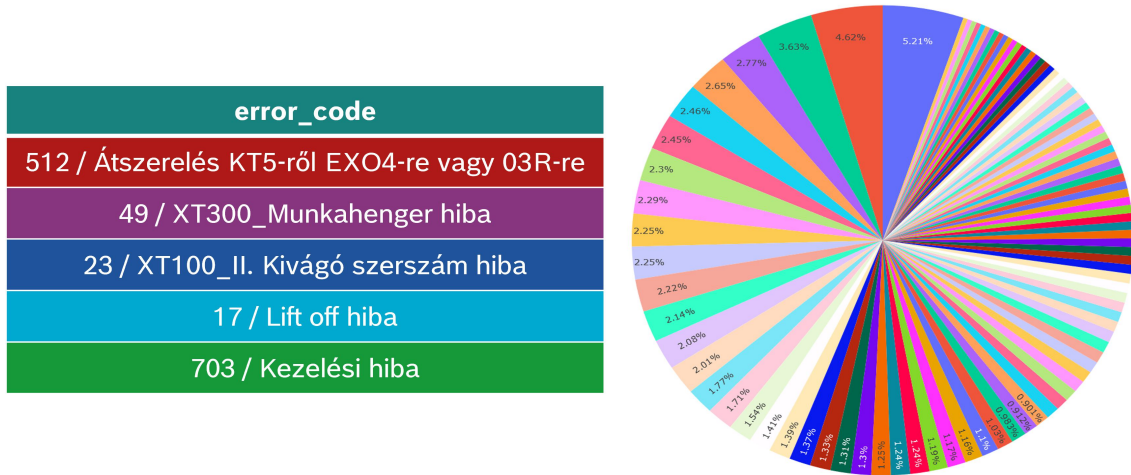


Figure 4.2: Examples and Distribution for "error_code" feature

Now let's discuss the most important feature in for this project, the "error_cause". Our research aims to investigate, process, augment and embed freely formulated text entries. Also, there is the predictive-generative Artificial Intelligence part, therefore we needed at least the penultimate set of information from the problem solving triplet, and given the fact, that the "error_code" feature is not freely formulated, we choose to develop the system using the data of the "error_cause" and check the predicted/generated answers based on the "action" feature. Of course, the preprocessing steps, such as spell checking, translation and even contextual embedding calculation will be applied on the "action" column to, but these are necessary to have multiple approaches for checking the system precision. That being said, we would like to present the investigation results describing the nature of the "error_code" entries, mainly the character length and the token-word (split-ted by space character) number of the texts:

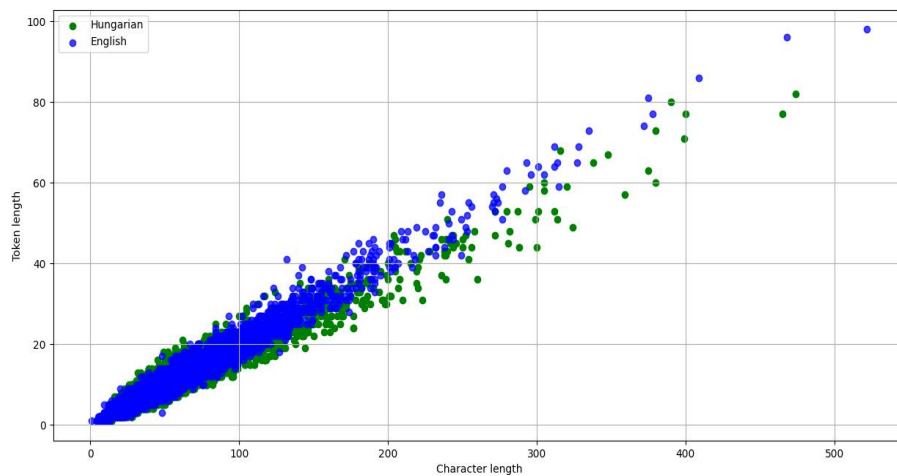


Figure 4.3: Character/Token length comparison for Hungarian and English texts

In our project, we began with the initial assumption that the Pareto principle, often referred to as the 80/20 rule, could be effectively applied to measure error occurrences in our industrial processes. The Pareto principle posits that roughly 80% of effects come from 20% of causes, making it a valuable tool in identifying and prioritizing problems in various contexts, including industrial problem-solving. By focusing on the most significant contributors to errors, we aimed to streamline our efforts in addressing issues and improving overall efficiency. This principle has been widely adopted in quality control and process optimization, allowing organizations to allocate resources more effectively and target interventions where they will have the greatest impact.

To further explore this concept, we investigated the "repair_time" feature, which captures the duration required to resolve different machine stoppages. Analyzing the data through a histogram revealed a clear distribution pattern: approximately 80% (78.23%) of the errors were classified as "light errors," as they were resolved quickly. In contrast, the remaining 20% (21.77%) of errors exhibited an exponentially decreasing distribution, indicating that they were more complex and time-consuming to address. This observation serves as compelling evidence supporting the validity of the Pareto principle in our context, demonstrating that a small subset of issues accounts for the majority of the repair time and highlighting the importance of prioritizing these critical errors for more effective problem-solving, but it still provides the opportunity to consider the more difficult-to-remediate errors that hinder the continuity of series production for the longest time.

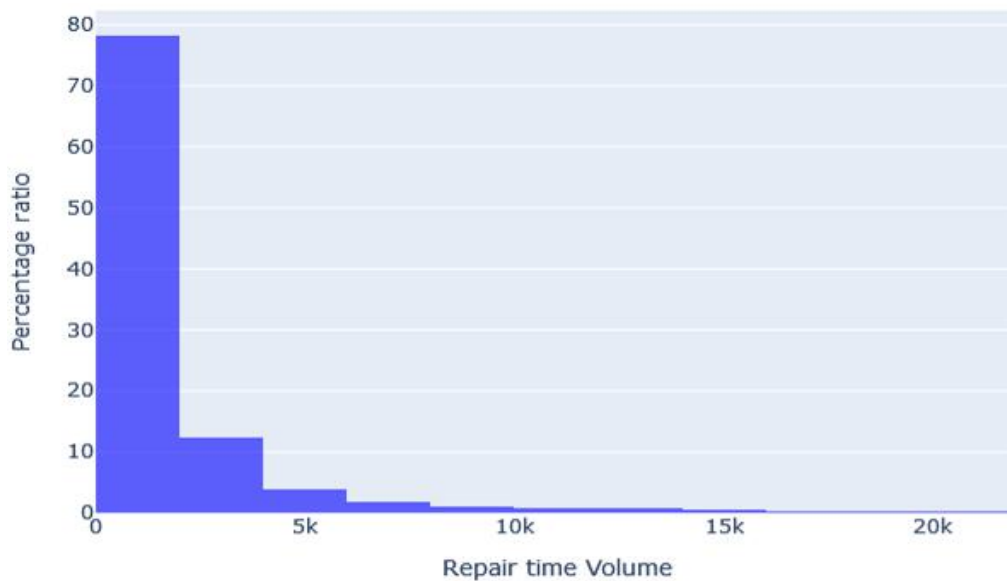


Figure 4.4: Repair time distribution of the machine stops

Chapter 5

Presentation of developed End-to-End system

5.1 Tools and Technologies used for the development

In this chapter, we will delve into the essential tools and technologies that have shaped our AI software development process. As we navigate the complexities of building intelligent applications, it is crucial to leverage the right development environments and platforms that facilitate efficient coding, collaboration, and experimentation. The landscape of AI software development is rich with resources that not only streamline our workflow but also enhance our ability to innovate and iterate. By understanding the strengths of each tool, we can better appreciate how they contribute to the overall success of our project.

Before diving into the specific development tools, it is essential to highlight the programming language we chose for our AI software development, which was **Python**. Renowned for its simplicity and readability, Python has become the go-to language for many in the fields of data science and machine learning. Its extensive ecosystem of libraries provides us with powerful resources for various tasks, including machine learning model development with libraries like Scikit-learn, which offers a range of algorithms and tools for model training and evaluation. For data visualization, we leveraged libraries such as Matplotlib and Pyplot, which enable us to create insightful graphs and charts that enhance our understanding of data trends. Additionally, Python excels in data handling with libraries like Pandas and Apache Spark, allowing us to efficiently manipulate and analyze large datasets. Furthermore, Python enjoys widespread support across major AI platforms, ensuring compatibility and ease of integration with tools like TensorFlow and

PyTorch. By choosing Python as our programming language, we positioned ourselves to take full advantage of these powerful libraries and frameworks, optimizing our development process, improving our overall productivity and final product quality.

To begin with, we utilized Visual Studio Code as our primary development environment, which offers a versatile and user-friendly interface for coding and debugging. This powerful code editor supports a wide range of programming languages and frameworks, making it adaptable to our specific project requirements. Its extensive library of extensions allows us to customize our workspace, integrating tools that enhance productivity and speed up our development process. Features such as IntelliSense provide intelligent code completion, while built-in Git support simplifies version control. Overall, Visual Studio Code has proven to be an ideal choice for comprehensive system development, enabling us to write, test, and refine our code efficiently.

For version control and collaboration, we turned to GitHub, which enables us to store and maintain different code versions through branching. This platform is essential for managing changes and facilitating teamwork, as it allows us to work on features independently without disrupting the main codebase. With GitHub, we can easily track modifications, review code through pull requests, and collaborate seamlessly with team members. The platform also provides robust issue tracking and project management tools, which help us prioritize tasks and monitor progress. By utilizing GitHub, we ensure that our development process remains organized and efficient, fostering a collaborative environment that enhances our productivity.

We have also integrated Hugging Face into our workflow for AI model research and benchmarking, as it offers a wealth of pre-trained models and resources that accelerate our experimentation and evaluation efforts. This platform provides easy access to state-of-the-art models for various NLP tasks, allowing us to leverage existing work and build upon it. Hugging Face also includes tools for fine-tuning models on our specific datasets, which enhances their performance and relevance. The community-driven nature of Hugging Face fosters collaboration and knowledge sharing, enabling us to stay updated with the latest advancements in AI research. By incorporating Hugging Face into our development process, we significantly enhance our capabilities in model training and evaluation, ultimately leading to more effective AI applications.

Lastly, we employed Databricks for experimentation, taking advantage of its Notebook feature for interactive coding and data analysis. This platform supports collaborative work, allowing multiple team members to contribute to the same project in real

time. Databricks excels in handling large datasets and provides robust tools for vectorization and data storage, making it invaluable for our AI model development. The integrated analytics engines, such as Apache Spark and Scala, allows us to run experiments, visualize results, and iterate quickly, which is crucial for refining our models. Also, it supports the integration and deployment of Ai models from Hugging Face. This was particularly important, because we had access to a large catalog of pretrained models and it also provided all the security measure for handling unknown resources preventing data leakage or malware attacks. The platform has provided the following computational capabilities:

Workload type	GPU instance	Memory
CPU		4GB per concurrency
GPU_SMALL	1xT4	16GB
GPU_MEDIUM	1xA10G	24GB
MULTIGPU_MEDIUM	4xA10G	96GB
GPU_MEDIUM_8	8xA10G	192GB

Table 5.1: Databricks Computational Capacity

Our Databricks Runtime Version was 16.2 ML (includes Apache Spark 3.5.2, Scala 2.12), the Worker type was Standard_DS3_v2 with 14GB Memory and 4 cores.

5.2 Goals and limitations of the workflow design

In terms of concrete implementation, this research project aimed to conduct an in-depth investigation and development for one underlined process steps of the Troubleshooting AI system. As an Applied Artificial Intelligence software, the goal and at the same time the measure of efficiency is revealed through the user experience, more precisely in the generation of messages that are as useful and, above all, professionally correct. In other words, it must be achieved that when the user, in our case the colleague responsible for the operation of the production lines, formulates a question or a problem description, the system generates answers or instructions that are as useful and as accurate as possible.

The development possibilities for a Text Generative AI application are vast and varied, each offering unique advantages and challenges. One promising avenue is the use of better training or base data, such as manually corrected datasets. By incorporating high-quality, curated data, the model can learn from more accurate examples, leading to improved output quality. However, this approach can be labor-intensive and costly, as it requires

significant human effort to curate and correct the data, potentially limiting scalability, not to mention the fact, that any time we would want to use the system at a new place (like different department of the factory), that would also come with new low quality input data which would have to be manually revised. This would really be the case of tilting at windmills.

Another possibility is the integration of more sophisticated models with a higher number of parameters. Larger models can capture more complex patterns in language, resulting in richer and more nuanced text generation. However, the trade-off is that these models demand substantial computational resources, making them expensive to train and deploy, and they may also suffer from diminishing returns in performance relative to their size.

A serious and advanced approach would be Targeted model training or fine-tuning presents another opportunity, where a model is trained from scratch or adapted to specific domains, such as legal, texts or in our case the inner technical specifications. This specialization can enhance the model's relevance and accuracy in niche areas, but it risks sacrificing generality, making the model less effective in broader contexts. Additionally, the fine-tuning process can be resource-intensive and may require extensive domain knowledge to ensure the model is appropriately trained, not to mention the reliability question in case we want to broaden the utilization area which would come with new specifications.

We could also use the technique of Reinforcement learning from run-time feedback, which is another innovative approach, where users can provide feedback on generated responses, allowing the model to learn from real-world interactions. This method can significantly improve the model's relevance and user satisfaction; however, it may lead to unreliable feedback if users are not representative of the broader audience or if their preferences are inconsistent.

An other tool would be Cross-model verification, which involves using multiple large language models to check and validate answers, and can enhance the reliability of generated content. By comparing outputs from different models, developers can identify discrepancies and improve overall accuracy. However, this approach can be computationally expensive and may complicate the deployment process, as it requires maintaining multiple models and ensuring their compatibility.

Last but not least, Intermediate algorithms, such as those that leverage attention mechanisms or statistical analytics can enhance the model's ability to process and generate text without the need for costly foundational model training. These algorithms are versatile and applicable across various inputs, making them a practical choice for im-

proving performance without the financial burden associated with training large models. Despite the practical advantages, they still have important limitations, such as performance Variability, given the fact that intermediate algorithms can vary significantly depending on the specific task or dataset. While they may excel in certain contexts, they might not generalize well across all applications. This variability can make it challenging to predict how well the algorithm will perform on different scenarios, potentially leading to inconsistent or even incomprehensible results.

Ultimately, while each of these development possibilities offers exciting potential for enhancing Text Generative AI applications, they also come with inherent challenges that must be carefully considered. Balancing the desire for improved performance with the constraints of cost, complexity, and generality is crucial in shaping the future of these technologies. Considering these possibilities, we choose to target the area of Intermediate algorithms for two reasons: from the perspective of the general project, the informational domain was narrowed to a manageable extent and from the perspective of the actual research, we intended to investigate and compare the results of different algorithm chains.

5.3 Presentation of the workflow

The most important feature of the developed workflow was that it was end-to-end. That is, during the processing of any arbitrary note, it can pass through all steps and the system reaches the point of generating a response. For this reason we have distinguished 5 major stages, with their respective sub-stages:

1. **Text Preprocessing:** The first step of the process was the text level preprocessing of the original Hungarian entries. The goal was to correct as many trivial text errors as possible with replacing or deleting unnecessary characters, using reg-exes for replacing or deleting sequences. Spell checking LLM-s were tried, but they had not presented any benefit to this part therefore this is entirely heuristic and manually created.
 - (a) Character replacement: Changing characters that have different coding than UTF-8 for preventing any later data error:
 - (b) Spell checking: Correcting simple and obvious spelling errors:
 - (c) Sequence cleaning: Changing character sequences with reg-exes, that were would have caused problems for later applied models:

Original	Corrected
"ö"	"ő"
"Ő"	"Ö"
"û"	"ű"

Table 5.2: Character Encoding Correction

Original	Corrected
"transzportt"	"transzport"
"talláltak"	"talláltak"
"kelett"	"kellett"

Table 5.3: Spell Checking word correction

2. **Translating:** given the fact that this project was working with Hungarian text but bot because of business and AI development reasons, we had to translate the entries to English.

- (a) **Model Loading:** To load and deploy a text translation model on Databricks, we begin by importing the model and tokenizer from the Hugging Face library. After loading the model, we deployed it for use within our Databricks environment. The used model had a **BLEU score of 38.03** and a **chrF-3 score of 61,37**, beating Googles contemporary model in both cases [14].
- (b) **Text translation:** We prepared our input data and with the help of Spark, we run the translation process for all the texts and saved the results to the database.
- (c) **Translation post-processing:** Because the translation models imperfections, some conspicuous and repeatable miss-translations were post-processed. This was also the case of manual rules settings:

Text in raw English translation	Text in post-processed translation
"losses"	"scrap parts"
"Outages"	"Scrap parts"
"fallout"	"scrap part"
"Discontinuation"	"Scrap part"

Table 5.5: Translation post-processing cases

3. Contextual Augmentation:

- (a) **Data Selection:** To understand the production context of the texts, it was necessary to retrieve the information related to the physical devices through the keys associated with the entries and collect it in the appropriate structure.

Regex Rule	Original	Corrected
<code>r",(?=[a-b]())' -> ", "</code>	"Kiesések (r8145,r7310,r8915)"	"Kiesések (r8145, r7310, r8915)"
<code>r'(?<= dl w)-l-(?= dl w)' -> "- "</code>	"Alkatrész elcsúszás: K17-A50-B55"	"Alkatrész elcsúszás: K17- A50- B55"
<code>r'(?! d[0-9])(' -> " ("</code>	"Pozícionáló tüske elha- jlott(8505)(8739)(8879)."	"Pozícionáló tüske elhajlott (8505) (8739) (8879)."

Table 5.4: Character Sequences Alterations

- (b) Text Merging: The contextual data texts and entries were linked to each other and than merged together at text level for the early fusion possibility.
4. **Embedding:** This project used the mechanism of contextual embeddings to analyze the semantic content of different posts. The state of the art representation generation is done with BERT model (Bidirectional Encoder Representations from Transformers) which is designed to pre-train deep bidirectional representations from unlabeled text by jointly conditioning on both left and right context in all layers [15]. Thanks to this, we were able to generate a numerical mapping from any input text, which was much easier to work with during subsequent analysis steps.
- (a) Model Loading: The loading and deployment of the embedding model also started with the importing the model and tokenizer from the Hugging Face library. After that, we deployed it for use within the same Databricks environment. The most important attributes of this model are the following: Language: English, Model Size: 434M params, Max Seq.: 8192, Length Dimension: 1024, MTEB-en: 65.39, LoCo:86.71 [16, 17];
- (b) Text Embedding: After data preparation of the input, again we utilized the abilities of Spark and we run process on the translated texts and saved the results to the cloud.
- (c) Embedding Combinations: Given the fact that we intended to investigate the differences between Early Fusion and Late Fusion, different vectorial operation had to be done between the embeddings of the raw entries and the independent embeddings of the augmentation texts.

5. Answer Selection:

- (a) Vector search or Clustering: Utilization of simple vector searching or clustering algorithms for finding the entri groups with similar meaning. The different

approaches come with different definitions of what is a best fitting group in specific case.

- (b) Result filtering: After finding the groups of any individual entri, we either have to set a distance limit or a statistical limit when we choose the right matches of an input entri. Once again, the limitation technique is dependent on how the groups were formed.

6. Answer Verification:

- (a) Model Utilization: For verification of the selected answer possibilities in such a large size, it was necessary to use PySpark for reaching the GPT4.o model through the API provided by OpenAI. For this purpose the Batch Inference method was used and ran on the Databricks platform.
- (b) Result Analysis: After all this was done, the generated results massages had to processed and the numerical values for the accuracy scores were calculated.

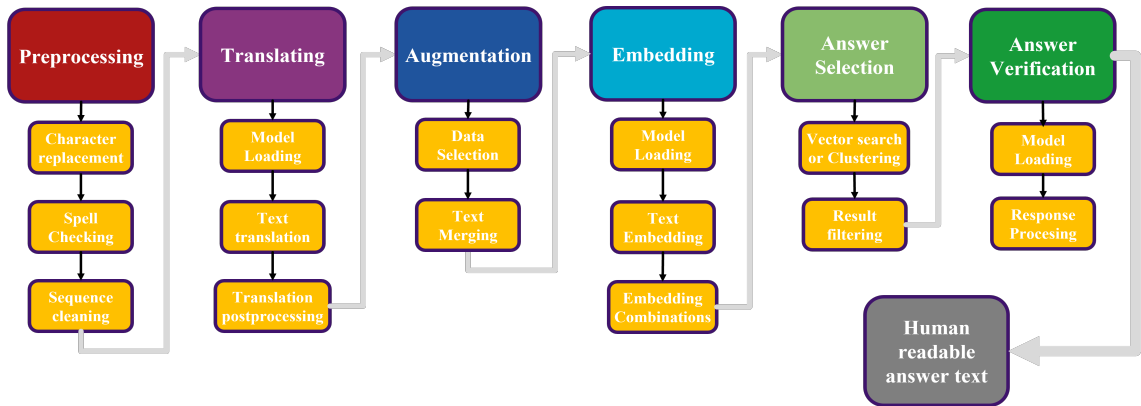


Figure 5.1: Workflow of the developed subsystem

The result of this whole process will be the precision score(s), which will incorporate the performance of the multiple possibilities used at different steps therefore expressing the ultimate accuracy of an entire combination.

Chapter 6

Text Augmentation

6.1 Introductions

Text augmentation is a vital technique in NLP (Natural language processing) that involves the systematic modification of text data to create variations, thereby enhancing the diversity of the training dataset. This process is particularly important in scenarios where data is limited, under-defined or imbalanced, as it helps to improve the robustness and generalization capabilities of machine learning models. By generating additional training examples, text augmentation mitigates the risk of overfitting, allowing models to learn more effectively from a broader range of inputs. Several well-established methods for text augmentation have emerged, each with its unique approach to enriching textual data.

One common technique is synonym replacement, where words in a sentence are substituted with their synonyms to create variations while preserving the original meaning. Another effective method is back-translation, which involves translating a piece of text into another language and then translating it back to the original language, often resulting in slightly altered phrasing that retains the core message. Random insertion is another technique, where new words are added to existing sentences, creating additional context and complexity.

These augmentation techniques are particularly useful because they allow models to encounter a wider array of linguistic structures and vocabulary, which enhances their ability to generalize to unseen data. As a result, large language models can calculate more accurate contextual embeddings, capturing the intricate relationships and nuances between words. This improved understanding of context is crucial for various NLP tasks, such as sentiment analysis, text classification, and machine translation, where the subtleties of

language play a significant role in performance[18].

Moreover, by training on augmented datasets, systems become more resilient to variations in language, such as colloquialisms or domain-specific jargon, which can further enhance their applicability across different contexts. Ultimately, text augmentation not only enriches the training data but also empowers models to deliver more reliable and contextually aware outputs. This makes it an essential step in the development of robust NLP applications, ensuring that they can effectively understand and generate human-like text in a variety of scenarios. As this field continues to evolve, the importance of text augmentation in training large language models will only grow, paving the way for more sophisticated and capable AI applications [19].

6.2 Incremental text augmentation

In our development of the text generative AI application, we focused on Incremental Augmentation, which involves adding new information step by step. This strategy enables us to manage complexity effectively while mitigating the risks associated with the Curse of Dimensionality. By incrementally incorporating information based on the importance assigned by the CUC framework, we can better analyze how each addition influences the system’s performance across various levels, ensuring a clear and manageable feature space. Attempting to explore all possible augmentation combinations within the framework would have required us to execute the entire process 2^{19} -times times, considering the complete set of 19 potential augmentation branches, not to mention the complexities introduced by different algorithms and their parameterization in subsequent steps. In this research we explored two primary methodologies: early fusion and late fusion.

Early fusion involves merging different text segments into a single sequence before processing, enabling the model to generate a unified contextual embedding that captures the relationships among the combined texts. This method can enhance the model’s comprehension of the overall context but may risk losing important nuances from individual segments. In our project, early fusion allows us to create a cohesive representation of the merged texts, facilitating a more coherent understanding for the model.

Conversely, late fusion processes each text segment independently, generating individual contextual embeddings before combining their vector representations. This approach preserves the unique contextual information of each text, which is crucial for tasks requiring a nuanced understanding of the content. After obtaining the separate embeddings, we

can combine them using various vector-level operations, such as multiplication, concatenation, or addition. For our project, we opted for addition, as it aligns with our goal of understanding the abstract operation of integrating new information. By employing late fusion, we can effectively leverage the strengths of both texts while minimizing the risks associated with high-dimensional data.

Here, we faced the challenge of setting up the augmentation information base. This limitation rally was a blocking point, because it was simply not, ultimately enhancing the performance of our text generative AI application to fill out the majority of the CUC branches. The maximum that we could achieve was the completion of the first two grand categories, namely **Machine** and **Material**. However, them being the two with the most branches, in total 10, a full combinatorial analysis was still not feasible (2^{10} -options is just not realizable). For this reason, we decided to go only with the first grand category, and even within that only with the simple incremental augmentation: adding chapters one by one without inner combination.

With this in mind, we generated embeddings of the two fusion methods and set up the following test metrics: the distance of the augmented entries from the original raw entries and the distance of the augmented entries from the augmentation text embedding. We used two distance metrics, namely Euclidean [20] and cosine similarity [21]. For the sake of clarity, here we only depicted measurements taken from 25 randomly selected entries in the generated figure (each measurement representing an augmentation case was based on the same 25 entries). Interactive analytics of the full dataset can be found in the appendix. Based on these, the following results were obtained from the early fusions approach:

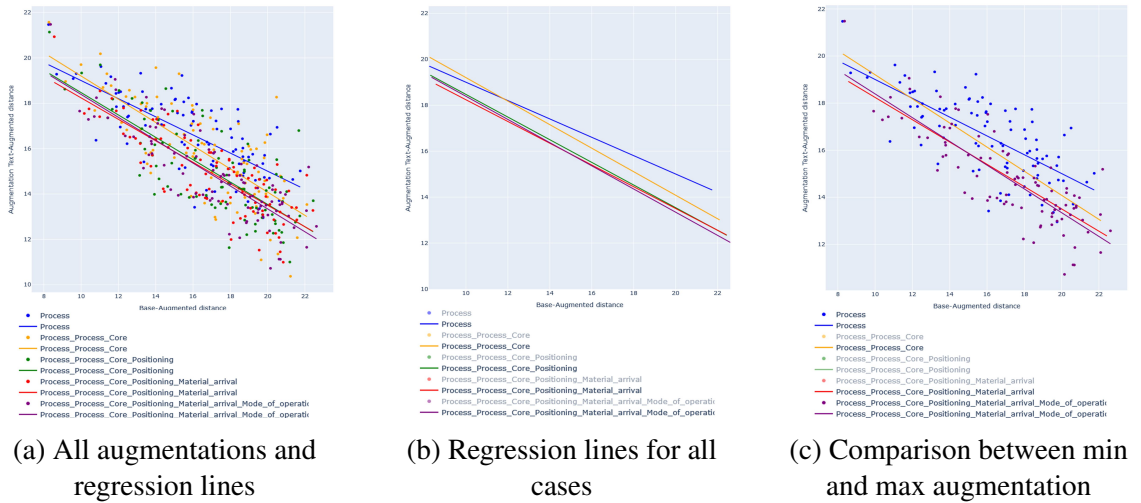


Figure 6.1: Augmented Embedding distances

Analyzing these graphs, we could draw multiple fundamentally important conclusions:

1. From figure 7.12a it is clearly stands out that the changes caused by the various contextual augmentations occur in a stable magnitude space for any given entry relative to the given reference points. This shows that these operations cause consistent impact on the given entries.
2. The next important phenomenon that emerges from the representation is that from the minimum to the maximum augmentation amount, the distance of the different sets from the reference points increases practically inversely proportionally to the augmentation magnitude, not to mention that even the order is preserved. Figure 7.12b clearly shows that, especially on the X-axis (Augmentation-Base distance).
3. The third important conclusion, highlighted on figure 7.12c, is that the relative changes in later steps during augmentation steps, such as from 4 (red) to 5 (purple), are much smaller than the relative changes in early steps, such as from 1 (blue) to 2 (yellow). In other words, the approximation of absolute contextual meaning slows down and becomes noisy. This basically proves the hypothesis that too much added content will eventually be contra-productive in achieving the main scope.

The same distance calculation experiment with the same entries was executed once more, but this time the distance metric was cosine dissimilarity. Here are the resulted graphs with the same setups as before:

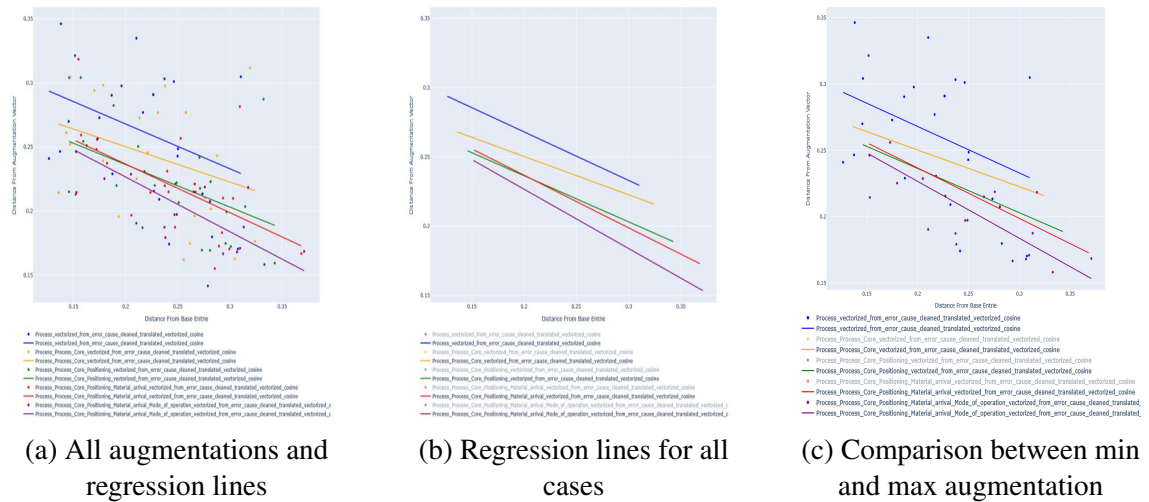


Figure 6.2: Augmented Embedding distances

Investigating the results, the partial conclusions are the same, but an important general conclusion stands out: the very same tendencies stand even with a different distance metric which by default measure different attributes.

After this experiment we ran the steps of the Late Fusion. This time the metric was only the cosine distance, because there is no point in investigating the euclidean distance changes of points after the addition operation (we would get back the vector sizes only), but the randomly selected 25 entries and investigation perspectives, included here and available in the appendix, were the same. Now lets investigate the results:

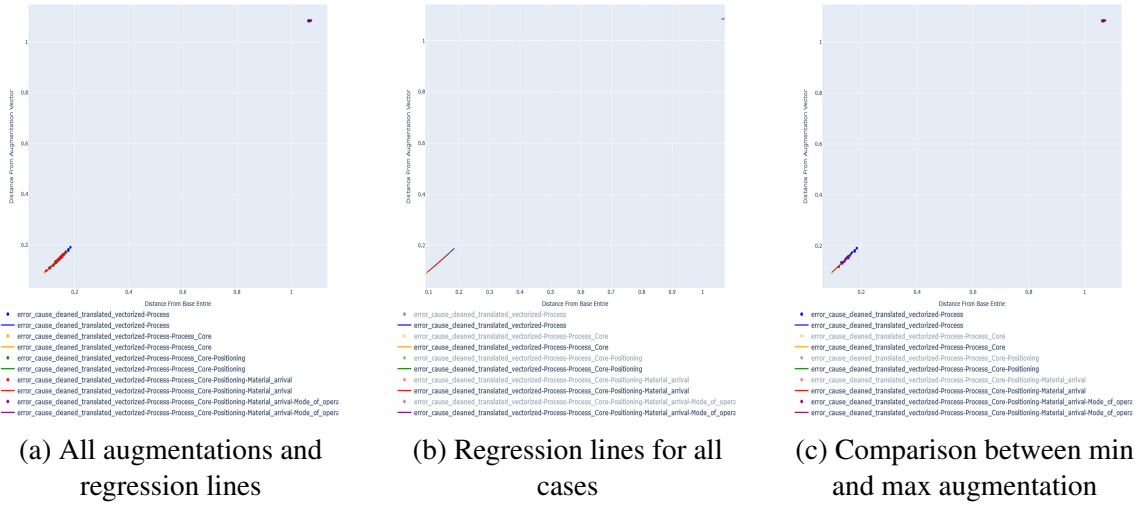


Figure 6.3: Augmented Embedding distances

Analyzing these results only on their individual level, the most relevant deductions are the following:

1. This augmentation technique yields unusable results, given the fact, that different groups are not separated from each other
2. The regression lines show no clear tendencies, meaning that from minimal to maximal there is no distance decrease, and there is no closing scissor at the more extensive augmentations.
3. The maximum augmentation has a huge and unexplainable jump from the rest of the cases. This could be a proof, that excessive additional information could result extreme behavior, but the jump from the penultimate category to that one is abnormally large.

Considering all the presented results and partial conclusions, the grand result of this chapter is, that the better augmentation technique from the two approaches is the **Early**

Fusion and considering the nature of the other ones results, there is no point in doing the later experimental steps.

Chapter 7

Text grouping

7.1 Introduction

In the domain of natural language processing, the ability to connect and categorize texts based on their semantic meaning is crucial. As we delve into the world of numeric embeddings, which transform textual data into a format that machines can understand, the challenge of grouping similar meanings becomes increasingly complex. While a straightforward approach might involve direct vector search techniques, such as identifying the closest k neighbors, this method often falls short in capturing the nuanced relationships between texts [22][23][24]. The inherent variability in language and the multifaceted nature of meaning necessitate a more sophisticated approach to clustering.

To address this challenge, we explore a variety of clustering methods, including Hierarchical clustering, KMeans, and DBSCAN. Each of these techniques offers unique advantages and insights into the structure of the data. Hierarchical clustering, for instance, allows for the creation of a tree-like structure that can reveal the relationships between groups at multiple levels of granularity. KMeans, on the other hand, excels in partitioning data into distinct clusters based on proximity, but it assumes spherical decision boundaries that may not always align with the true distribution of meanings. Meanwhile, DBSCAN provides the flexibility to identify clusters of varying shapes and densities, making it particularly useful in scenarios where the nature of decision boundaries is unknown.

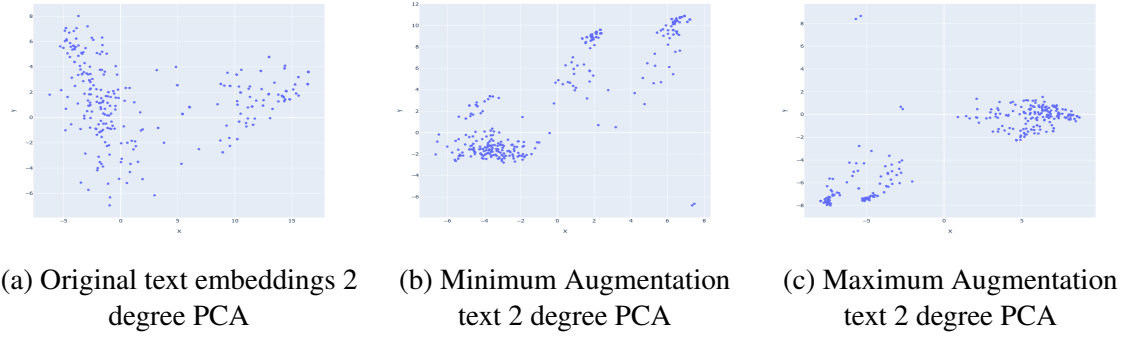


Figure 7.1: Visible differences between the same 1000 text embeddings at different augmentation level

The necessity of experimenting with these diverse clustering methods lies in the unpredictable nature of language itself. Meanings can overlap, diverge, and exist in shades of ambiguity, making it essential to adopt a multifaceted approach to uncover the underlying patterns within the data. By employing a combination of clustering techniques, we can better navigate the complexities of textual embeddings and enhance our ability to identify groups with similar meanings. This exploration not only enriches our understanding of the data but also paves the way for more effective applications in information retrieval, recommendation systems, and beyond. Ultimately, the quest for meaningful connections in text data is a journey that demands both creativity and rigor, as we strive to illuminate the intricate tapestry of human language.

7.2 Hierarchical clustering

7.2.1 Approach Description

Hierarchical clustering is a popular unsupervised machine learning algorithm used to group similar data points into a hierarchy of clusters. This method builds a tree-like structure known as a dendrogram, which visually represents the relationships between the data points based on their similarities. Hierarchical clustering can be performed using either an agglomerative approach, where each data point starts as its own cluster and merges them iteratively, or a divisive approach, which begins with a single cluster and splits it into smaller ones. This algorithm is particularly useful for exploratory data analysis, as it allows for the identification of natural groupings within the data without requiring prior knowledge of the number of clusters.

Investigating hierarchical clustering for text embeddings is essential due to the complex and often ambiguous nature of semantic relationships in language. Unlike traditional clustering methods that assume well-defined, spherical decision boundaries, hierarchical clustering allows for a more detailed exploration of how different meanings can overlap and diverge [25]. The unique aspect of this algorithm lies in its ability to create a dendrogram, which not only reveals the hierarchical structure of the data but also provides insights into the varying degrees of similarity among text embeddings. This is particularly important in natural language processing, where meanings can exist on a spectrum rather than in discrete categories. The decision boundaries in this context may be irregular and multifaceted, reflecting the intricacies of human language and thought. By employing hierarchical clustering, we can better capture these subtleties, enabling a more accurate grouping of similar meanings and enhancing our understanding of the underlying relationships within the data. This exploration ultimately aids in refining our approach to text analysis and improves the effectiveness of subsequent applications.

In our project, we utilized the linkage function from the `scipy.cluster.hierarchy` library to perform hierarchical clustering on our data. This function has two important parameters: the metric, which we set to `'euclidean'`, reflecting our assumption that closely related meanings would have small Euclidean distances between their embedding vectors. The second parameter, `method`, allows us to choose how the distance between clusters is calculated, and we experimented with several options: `'single'`, `'complete'`, `'average'`, and `'ward'`. The `'single'` method calculates the distance between the closest points in two clusters, which can lead to chaining effects where clusters may be formed based on a few close points. The `'complete'` method, on the other hand, considers the distance between the farthest points in two clusters, which can result in more compact clusters. The `'average'` method computes the average distance between all pairs of points in the two clusters, providing a balanced approach. Lastly, the `'ward'` method minimizes the total within-cluster variance, which tends to create clusters of similar sizes and shapes. By exploring these different linkage methods, we aimed to identify the most effective approach for clustering our data based on the relationships inherent in the embedding vectors.

7.2.2 Test Results

For investigating the best option for the linkage method, we generated the dendrograms for the raw `entri` embeddings and all the augmented `entire` embeddings as well. In this

thesis we included the resulted plots only for the raw embeddings but the others are all available in the appendix. Now lets check the dendrograms:

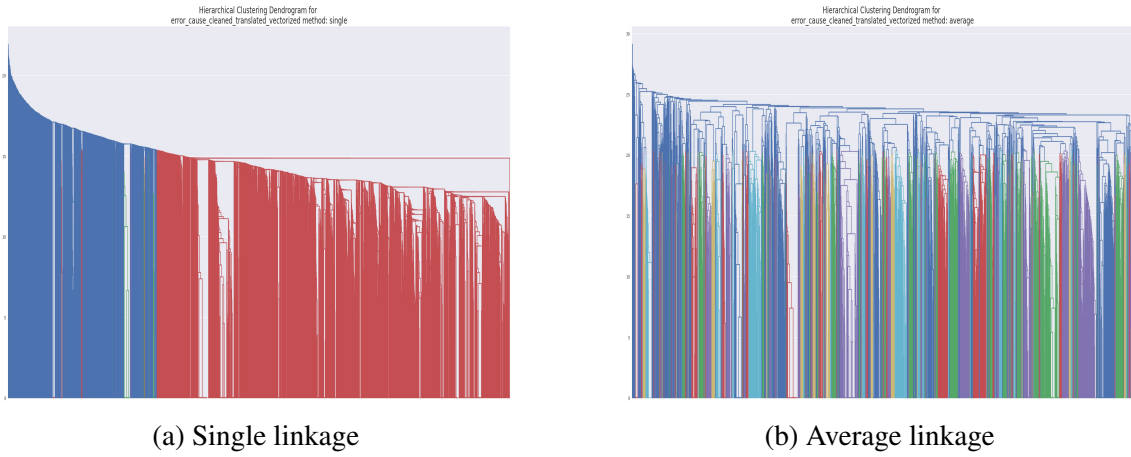


Figure 7.2: Single and Average linkage

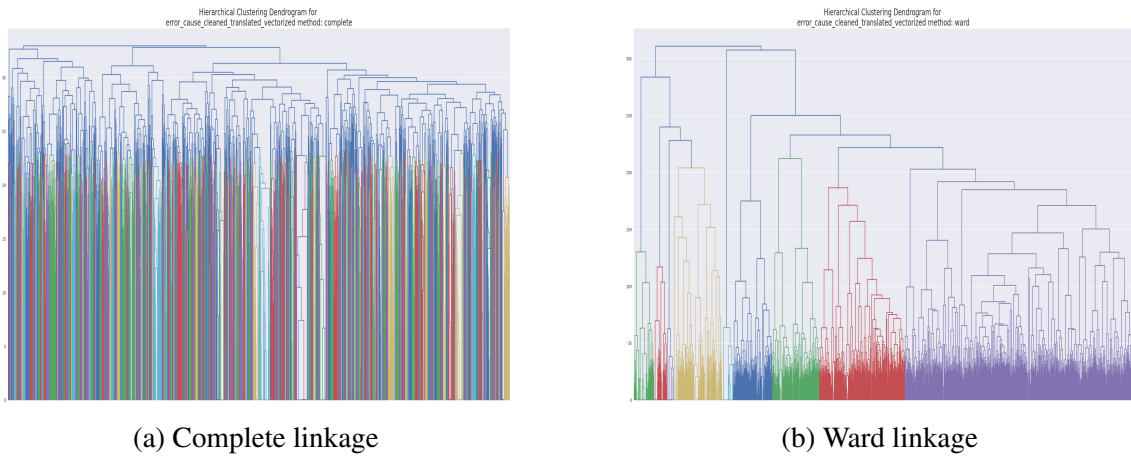


Figure 7.3: Complete and Ward linkage

We can conclude the following immediate lessons after visually inspecting the dendrograms:

1. The 7.12a dendrogram shows a chained structure, where clusters merge progressively at very low distances. This often leads to the "chaining effect", where clusters form long chains rather than compact groups. This method may not be ideal for your case because it suggests that some text embeddings are loosely connected rather than forming distinct groups.
2. The 7.12b dendrogram appears very dense with deep hierarchical structures. The branches are well-balanced, but there is a lot of fine-grained fragmentation. This

suggests that average linkage maintains relatively uniform distances between clusters, avoiding extreme merging but potentially leading to many small clusters.

3. 7.12c is similar to the average linkage, but with more pronounced separation at the top of the hierarchy. The overall structure looks slightly cleaner, meaning clusters are formed in a way that avoids large intra-cluster variations. This could be useful when you want compact clusters with maximum intra-cluster similarity.
4. The clusters on 7.12c are more structured and balanced, with clear large groupings at higher levels. This suggests it minimizes the variance within clusters, leading to well-defined and stable clusters. Ward's method is often preferred when seeking clear, distinct clusters, making it a good choice for textual embeddings.
5. **General Conclusions:** Ward's method appears to be the best for obtaining well-separated clusters, which is important when working with contextual embeddings of free text. Single linkage is likely not suitable due to the chaining effect, which could lead to artificial long clusters rather than meaningful groupings. Complete and average linkage are viable but may create too many fragmented clusters. The results of the non augmented embeddings are consistent with the results of the augmented ones, showing that the ward linkage provides the best clustering based on the dendograms.

Before we went further, we wanted to perform another inspection, which presented objectively quantifiable attributes. We choose to calculate the silhouette measure for different cluster sizes. The reason for this was that the necessary output of this step was the multiple groups of similar responses, and this measure could show what is the right fragmentation point of the whole dataset. In addition, this score is calculable for other clustering algorithms and provides a robust base for direct comparison, considering that it really shows how well the embeddings are separated.

For this reason we calculated this score for cluster number from 2 to 50 and generated the plots:

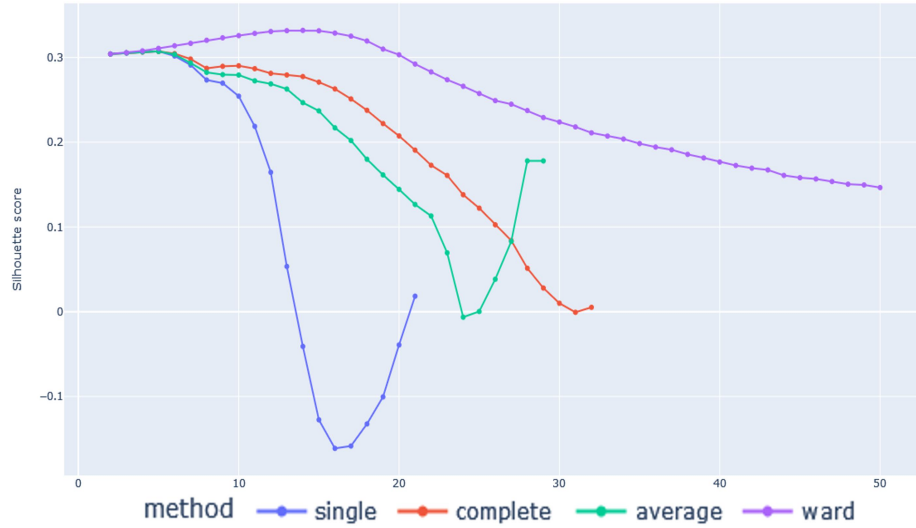


Figure 7.4: Silhouette score for raw text embeddings

According to this investigation, we have reached to the following outcome:

1. Ward's method provides the best clustering results out of the four possibilities across all cluster number.
2. The highest silhouette measure was reached relatively early in terms of the cluster numbers ($k = 14$) and the value itself was relatively low (0.3319).
3. **General Conclusions:** The best fitting linkage method calculated by this measure is consistent with all the augmented embeddings to. In all cases, ward method performed the best, with relatively small cluster number and silhouette score.

7.2.3 Final conclusions

Before going to the next step of this project, we have selected the top 5 cluster numbers with the highest silhouette score for all embedding types. The outcomes of this study are detailed in the following section.

Table 7.1: Hierarchical Silhouette Score

Entrie / k	1.	2.	3.	4.	5.
Base	14 (0.3319)	13 (0.3316)	15 (0.3315)	12 (0.3306)	16 (0.3287)
Aug. 1	8 (0.3046)	9 (0.3035)	7 (0.3003)	6 (0.2978)	10 (0.2974)
Aug. 5	7 (0.3008)	6 (0.3007)	8 (0.2983)	5 (0.2959)	4 (0.2909)

7.3 KMeans clustering

7.3.1 Approach Description

The KMeans clustering algorithm is a widely used unsupervised machine learning technique designed to partition data points into distinct groups based on their similarities. This algorithm operates by initializing a specified number of centroids, which represent the center of each cluster. The data points are then assigned to the nearest centroid, and the centroids are recalculated based on the mean of the assigned points. This process is repeated iteratively until the centroids stabilize, resulting in well-defined clusters. KMeans is particularly effective for large datasets and is favored for its simplicity and efficiency, making it a popular choice for various applications in data analysis [26].

Exploring this clustering approach for text embeddings is crucial due to the need for clear and distinct groupings in semantic analysis. This method assumes that clusters are spherical, which can be beneficial when the data exhibits such characteristics. However, the flexibility of KMeans also allows for experimentation with different numbers of clusters, enabling researchers to uncover patterns that may not be immediately apparent. The algorithm's reliance on distance metrics, typically Euclidean, makes it suitable for capturing relationships among embedding vectors, although it may struggle with irregularly shaped clusters. By applying KMeans, we can gain insights into the underlying structure of the text embeddings, facilitating a more organized approach to understanding semantic similarities and differences.

In our study, we implemented the KMeans algorithm using the KMeans class from the scikit-learn library, a robust tool for machine learning in Python. The primary parameters we configured included the number of clusters, which we determined through methods such as the elbow method to identify the optimal k value. Additionally, we set the `random_state` parameter to ensure reproducibility of our results. The KMeans algorithm also allows for the specification of initialization methods, such as `'k-means++'`, which helps in selecting initial centroids that are more likely to lead to better clustering outcomes. By leveraging these features, we aimed to effectively categorize our text embeddings, enhancing our ability to analyze and interpret the semantic relationships present in the data.

7.3.2 Test Results

As mentioned before, the state of the art approach for determining the ideal cluster number for KMeans is the Elbow method that uses the Within sum group of squares metric, also known as inertia, for calculating the quality of the clustering. With this method we search for the cluster number setting with a significant drop from the previous number and no notable decrease after that. For this reason, consistently with the investigation of the Hierarchical clustering, we have generated this score for cluster numbers 2 to 50 for the raw text embeddings and for the 5 incrementally augmented embeddings as well. Here are the results:

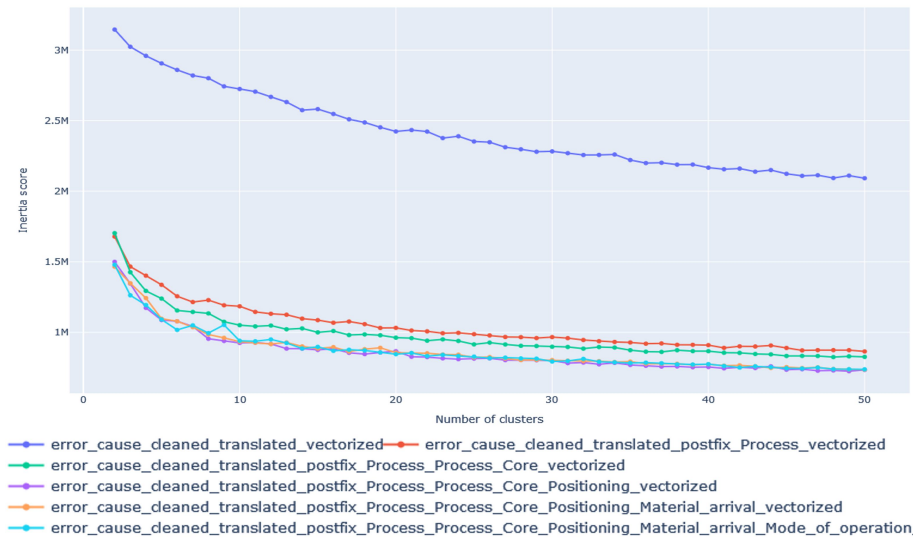


Figure 7.5: KMeans Clustering Inertia Score

Looking at the figure, it is immediately clear that there is no characteristic jump, elbow on the curve. As a result of this it is necessary to calculate the differentiated series of inertia change. A differentiated series should be interpreted as calculating the percentage change between successive values. Here is the result plots:

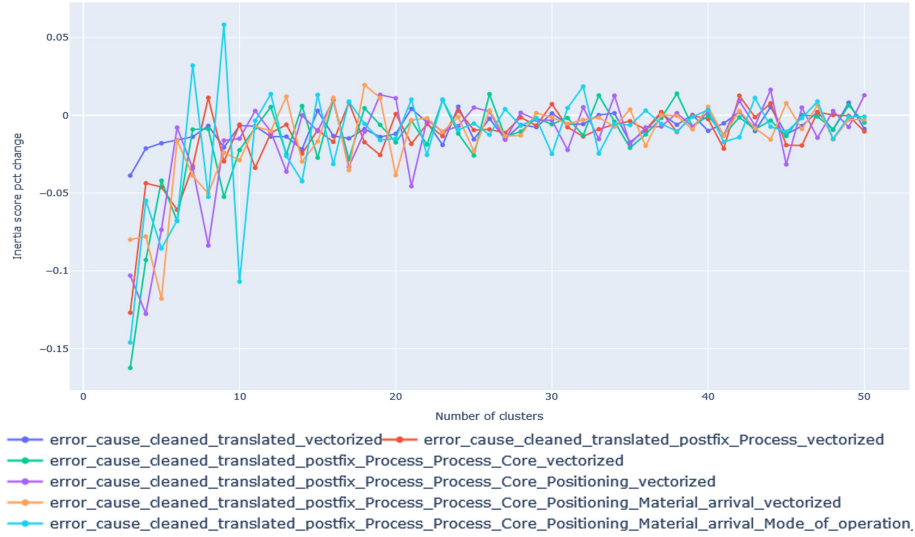


Figure 7.6: KMeans Clustering Inertia Score Change

According to this investigation, we have reached to the following outcome:

1. The curves show a steep decline initially (for low k values) and then gradually flatten, implying that increasing clusters beyond a certain point yields diminishing returns.
2. However, the proportionality of the change is not much smaller than the initial increase in the number of clusters, considering that it is important to divide the inputs into as many meaningful groups as possible. It is therefore worth examining the grouping with a larger k .
3. Different features show different inertia trends, but the overall pattern is similar. This implies that while feature selection impacts clustering efficiency, the general trend of diminishing inertia reduction holds across different feature sets.
4. The blue curve (raw text embeddings) has consistently higher inertia values compared to the others and in the percentage change graph, the blue curve fluctuates less compared to the others. This could indicate that this particular feature set struggles to form compact clusters, has a less clear cluster structure and it might contain noisy data. This suggests that it might contain noisy data, making it less effective for KMeans clustering.

In the case of K Means clustering, the silhouette score can also be an important metric in establishing the resulting groupings. This is also specifically needed so that we can directly compare it with the Hierarchical and then DBSCAN algorithms. Similar to the

inertia score, this time we calculated the pons score associated with cluster numbers from 2 to 50 and its differentiated series. The results are shown in the following figures:

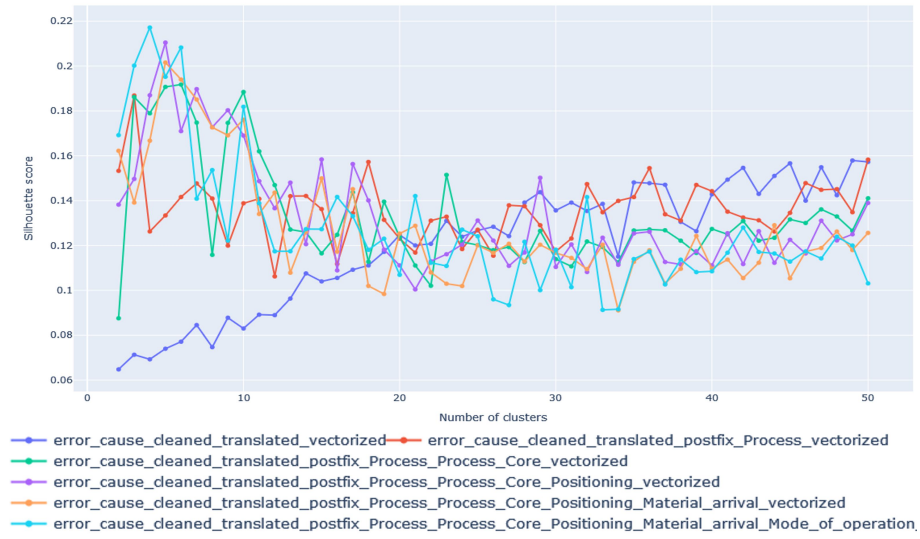


Figure 7.7: KMeans Clustering Silhouette Score



Figure 7.8: KMeans Clustering Silhouette Score Change

From these results we could draw the following conclusions:

1. There is a sharp contrast between the blue curve and the other groups. While the blue curve shows a continuous improvement trend with increasing cluster numbers, the others quickly drop from the high score at the initial low cluster number, but then level off at a certain level.
2. However, changes occurring with increasing k fluctuate within a narrow interval in the case of the blue curve, while in the other cases more serious fluctuations can be

experienced, even in a negative direction (worsening in the quality of the clustering despite increasing the cluster number).

7.3.3 Final conclusions

For determining the best setting for the KMeans clustering, considering that we should look beyond the direct values of the investigated measures, considering that the individual metrics do not describe the change in the quality of classification, we decided to combine them, namely, we add up the change metrics and based on this select the top five k cluster number setting. Furthermore, we set another rule that the k value to be selected must be at least 5, because it necessary for the answer generation step to split up the data to a necessary level, otherwise the noisy and overlapping nature of base information will make it impossible to reach a reasonable level of distinction. Results of the investigation is as follows:

Table 7.2: KMeans Change Score

Entrie / k	1.	2.	3.	4.	5.
Base	35 (26.98%)	9 (15.50%)	40 (12.01%)	49 (11.67%)	28 (11.44%)
Aug. 1	13 (33.08%)	17 (21.04%)	27 (18.34%)	32 (18.34%)	50 (16.35%)
Aug. 5	32 (41.46%)	10 (38.14%)	21 (33.87%)	28 (29.53%)	35 (23.79%)

7.4 DBSCAN clustering

7.4.1 Approach Description

DBSCAN, or Density-Based Spatial Clustering of Applications with Noise, is an effective unsupervised machine learning algorithm that identifies clusters based on the density of data points in a given space. Unlike methods that require the specification of the number of clusters beforehand, this approach groups points that are closely packed together while marking points in low-density regions as outliers. The algorithm operates by defining a neighborhood around each point using two key parameters: **eps** (epsilon), which specifies the maximum distance between two points for them to be considered part of the same cluster, and **min_samples** (minimum neighbor), which indicates the minimum number of points required to form a dense region. This characteristic makes DBSCAN particularly advantageous for datasets with varying shapes and sizes, as it can effectively identify clusters of arbitrary form[27].

Determining the optimal values for the `min_samples` and `eps` parameters poses a significant challenge, as it necessitates prior knowledge about the underlying structure of the data. Setting these parameters incorrectly can lead to either overly fragmented clusters or the merging of distinct groups. This complexity highlights the importance of careful tuning to achieve meaningful results. In the following section, we will introduce a developed algorithm designed to assist in selecting these parameters more effectively, thereby enhancing the clustering process.

The motivation for experimenting with this density-based clustering method stems from the need to account for the potential presence of non-convex cluster structures and complex decision boundaries among different semantic embeddings. Traditional clustering algorithms often assume that clusters are convex and well-separated, which may not accurately reflect the intricacies of language and meaning. By utilizing DBSCAN, we can effectively identify clusters that may have irregular shapes (non convex) and varying densities, allowing us to capture the true relationships between text embeddings. This flexibility is particularly important in natural language processing, where meanings can overlap and exist in nuanced forms, making it essential to adopt a method that can adapt to the complexities inherent in the data.

For our analysis, we utilized the DBSCAN implementation from the `scikit-learn` library, which provides a straightforward interface for applying this clustering technique. The primary parameters we focused on were `eps` and `min_samples`, which we adjusted based on our exploratory analysis of the data. Additionally, we employed the `metric` parameter to specify the distance measure, opting for `'euclidean'` to align with our embedding vectors. The algorithm's ability to identify noise points also proved beneficial, as it allowed us to filter out irrelevant data that could skew our results. By applying DBSCAN, we aimed to reveal the inherent structure within our text embeddings, facilitating a deeper exploration of the semantic relationships present in the dataset.

7.4.2 Hyperparameter Search

As mentioned before, generally speaking it is necessary to have prior knowledge about the structure and the nature of the input data before setting up the hyperparameter of the algorithm, otherwise it could lead to absolutely blind guessing[28]. For this reason, we came up with the following algorithm:

1. We have calculated the distances of the 25 closest neighbors for every data point.

Then we have organized these distances in an ascending order for every neighbor group, as 1st neighbor distances separately, 2nd neighbor distances separately and so on. We indexed these values from 0 to 1, with the smallest k-neighbor distance assigned to index 0 and the largest k-neighbor distance assigned to index 1.

2. We applied the Gaussian Kernel Density Estimation to generate the distribution curves of these distances from 0 to 25 (this because, this was the upper limit of the occurring distances) and have calculated the the local maximums of the density values with the additional constraint of reaching the 5% percent (0.05 normalized) distribution limit in case of every neighbor group.
3. And lastly, we have generated the surrounding distance of the all local maximums for all neighbor groups. The way it was determined is as follows: we calculated the distance range which we got by multiplying the upper distance limit and the occurrence probability of the local maximum value, than subtracted and added to the the value of the local maximum element and lastly, we selected a given number of evenly spaced samples from resulted interval that were not less than 0 and less than or equal to 25. We proceeded even if we got less samples than the given number.

Here are the plots for every step of the developed algorithm ran on the embeddings of the base texts:

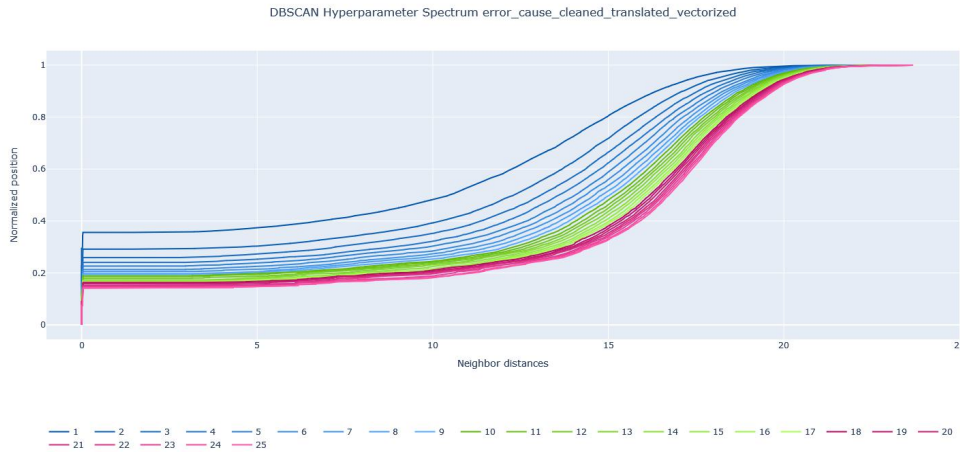


Figure 7.9: Step 1: Neighbor Distance Spectrum Base Texts

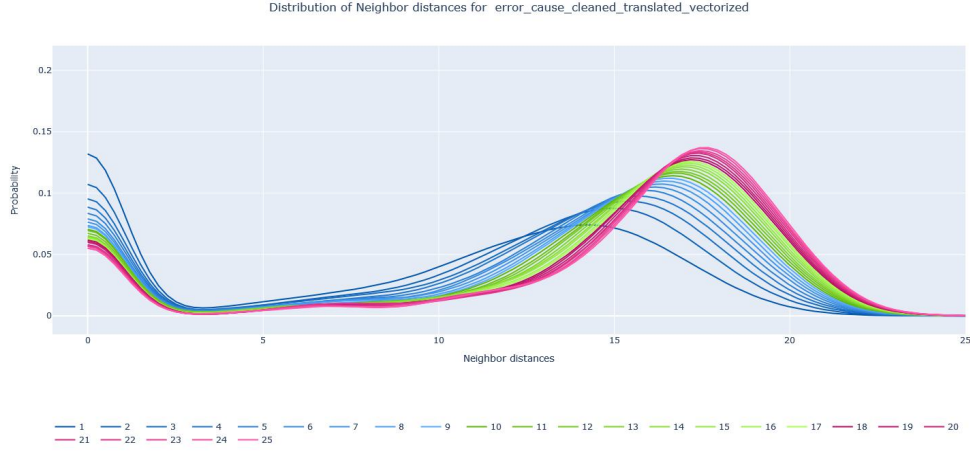


Figure 7.10: Step 2: Neighbor Distance Distribution Base Texts

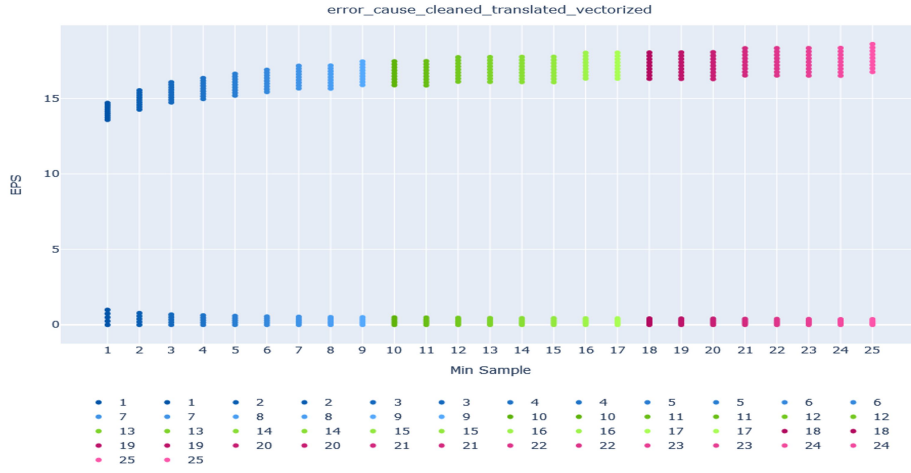


Figure 7.11: Step 3: Hyperparameter Field Base Texts

Looking at the figures depicting the steps, it turns out that basically from the 1 neighbor to the 25 neighbor group, the distance spectrum takes on a regular shape, the higher the ordinal number, the greater the proportion of real distance values (not 0), they converge in parallel to each other at the maximum distance value 7.9. The distribution 7.10 curve shows that basically there are 2 focal points, the first close to 0, the second at two-thirds of the maximum distance. The consequence of this is shown at 7.11, that the final parameter field basically takes on two band ranges. These characteristics can also be observed in the case of augmented embeddings, with the difference that the second degree points are found at a smaller distance value, at the third of the maximum distance, and the higher the augmentation level, the more this value decreases.

7.4.3 Test Results

In evaluating the performance of the DBSCAN algorithm, we employed the silhouette score as our primary validation metric. This measure is particularly suitable for density-based clustering, as it assesses how well each data point is clustered relative to its neighbors. Given the potential for non-convex cluster structures inherent in DBSCAN, the silhouette score provides valuable insights into the quality of the clustering by considering both the cohesion within clusters and the separation between them. To conduct our analysis, we calculated the silhouette scores for various parameter settings, specifically focusing on different combinations of the `eps` and `min_samples` values. This approach allowed us to identify the most effective configurations for our text embeddings. Here are the results:

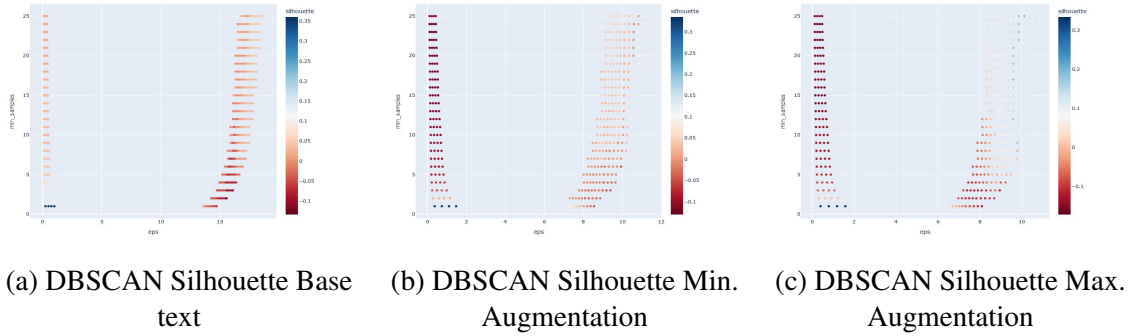


Figure 7.12: Single and Average linkage

By inspecting the generated images, there are a couple of conclusions that are worth to be mentioned:

1. In all cases the best silhouette score was reached at low `eps` band and minimal `min_sample` setting.
2. There is an opposite trend between the two bands of hyperparameters. In case of the low `eps` band, the greater the `min_sample` the lower the silhouette score. In case of the large `eps` band, the greater the `min_sample` the higher the silhouette score.
3. In case of the base embeddings, higher silhouette score was reached at the low `eps` band than at the large `eps` band across all `min_sample`. But in case of the augmented embeddings, the large `eps` band has higher silhouette score with the large `min_sample` than the low `eps` band.

4. In case of every embedding type, it occurred that the very same silhouette score was achieved with different but close eps setting for the same min_sample. For avoiding unvaried hyperparameter setting, these cases were combined and the final eps was average of the group.

7.4.4 Final conclusions

Based on the upper conclusions, we have selected the top 5 hyperparameter settings for all the embedding cases. We had one special rule beside the merging the same min_sample and close eps cases into one, which was to select a different setting as the fifth member, in case the top five was very similar to each other. The findings from our analysis are presented below:

Table 7.3: DBSCAN Silhouette Score

Entry / eps-min_sample	1.	2.	3.	4.	5.
Base	0.620 - 1 (0.3606)	0.486 - 2 (0.1417)	0.425 - 3 (0.0862)	0.391 - 4 (0.0681)	18.60 - 25 (0.0525)
Aug. 1	0.921 - 1 (0.3346)	10.52 - 13 (0.1296)	10.53 - 16 (0.1259)	10.58 - 18 (0.1177)	7.10 - 1 (0.1135)
Aug. 5	0.986 - 1 (0.3340)	10.120 - 25 (0.186)	9.85 - 12 (0.1785)	9.857 - 24 (0.1781)	9.793 - 9 (0.1767)

Chapter 8

Answer Selection Verification

8.1 Introduction

In this study, we aimed to develop a system for grouping text entries based on their semantic similarities. The ability to effectively categorize texts is crucial in various applications, such as information retrieval, recommendation systems, and content analysis. Given the complexity of language and the nuances of meaning, it became necessary to compare different methods for identifying similar texts. By evaluating various approaches, we sought to determine which techniques would yield the most accurate and meaningful groupings for our specific use case.

To achieve this, we explored several answer selection methods that could guide the identification of similar text entries. One approach involved setting distance limitations, where only texts within a certain proximity to a given input were considered relevant. Another method focused on limiting the number of potential matches, ensuring that only the top candidates were selected for further analysis. Additionally, we considered logical constraints, such as maintaining semantic coherence within clusters, which could enhance the quality of the results. These selection criteria were essential in refining our search for similar texts and ensuring that the most relevant entries were prioritized.

The go to approach is simple vectorial searching. This builds up the searching mechanism around the input and tries to find matches from the established cloud. This has two major disadvantages: first, this is a very slow mechanism, considering that it needs a lot of distance calculation operations to find the appropriate elements, and second, this is a very rigid filtering concept since we have no clue what are the real meaning organizing rules of the embedding generations. For example, something might have a better absolute distance

from the input, but along certain embedding axis, that reflect the real meaning, it is diverged. And if we consider, that these embeddings have huge dimensionalities and due to the imperfection of the model, their order of importance is inconsistent, vector searching might not have the best filtering ability, implying the need to check other techniques as well. Considering the fact that the goal of the search is not necessarily similarity with individual independent elements, but rather with sets of specifically similar characteristics, and only then does the selection of individual elements from these sets follow, clustering is an intuitive choice for alternative search algorithms.

Starting with hierarchical clustering method, we implemented a combination of distance and cluster coherence criteria. We examined the dendrogram to identify clusters that exhibited strong semantic relationships while also adhering to distance limitations. This dual approach enabled us to select entries that not only belonged to the same cluster but also maintained a meaningful proximity to the input text. By integrating these selection rules, we aimed to maximize the effectiveness of our clustering efforts and ensure that the resulting groupings accurately reflected the underlying semantic structures present in the data.

When applying the KMeans clustering algorithm, we utilized a distance-based selection method to identify the k closest entries to the nearest centroid. This approach allowed us to focus on the most relevant texts, as the algorithm inherently grouped similar entries around defined centers. By selecting the closest options, we ensured that the results reflected the underlying structure of the data, providing a clear representation of semantic relationships. This method proved effective in generating meaningful clusters, enabling us to retrieve relevant texts efficiently.

In contrast, when employing the DBSCAN algorithm, we adopted a different strategy for answer selection. Instead of solely relying on direct distances, we considered the logical constraint of remaining within the same cluster, even if some entries were further away in terms of Euclidean distance. This approach allowed us to capture the density-based nature of the algorithm, ensuring that we included texts that shared similar meanings despite potential variations in their distances from the input. By prioritizing cluster membership over strict distance metrics, we enhanced the robustness of our results.

8.2 Evaluation metrics

As mentioned before, the two limitation of selection are the distance and the number of items. This question is complicated to answer given the fact, that the different searching algorithms would be differentially affected by yet two new hyperparameter. Not to mention the goal of this thesis, which intends to compare the results of different approaches as directly as possible. And if we start optimizing on these new aspect, we loose from the robustness of the comparison. Therefore, we have decided to go with the following analysis: the primary factor was the maximum number of selected items which was set to **10** (the closest 10 option by direct distance withing the allowed searching area) together with a maximum distance of **25** (based on the DBSCAN hyperparameter search sections neighbor spectrum distance 7.9) , than we have averaged the distances of the results and calculated the the distribution and the ordered series of these values. Based on the distribution we could see how different approaches perform in general, and by the ordered series we can see which option gives the best results, which option gives the second best results and etc. for the whole of the test data.

Together with this, it was important to take into consideration the user experience part of the system, such as how was the average usefulness of the included option or did they even receive any correct answers? For this reason we have decided to measure the systems accuracy with two metrics: the ratio of the accepted answers or **Recall** and absolute measure or **Pass Measure** which is 1 if any correct answer was found otherwise 0. This is important because it is key to return a transparent and humanly processable quantity to the user, preferably including only the truly correct answers, but it is also important not to filter out at least one correct answer that can be returned. In case of rarer manufacturing error event if an answer is not numerically as close as it would be for more common error cases, it could still be relevant to a human [29].

For this test phase we used a LLM model, namely Gpt-4o. This was necessary, because the available dataset is still unlabeled and for a thorough investigation requires huge number of test cases, in our case it was 1084 instances for all the different augmentation and algorithm setups. Obviously, for this step we have came up with a and temperature setting, which **zero**, and special prompt to retrieve as consistent results as possible. For this purpose we have experimented with multiple evolutions of the input prompt. Because the accuracy of this investigation mechanism was in particularly important, we have ran a 20-item test for each prompt evolutions and than investigated the results manually [30].

The first variant designed to formulate the result as a simple floating point number, as the ration of accepted answers and the total answers. The prompt is as follows:

Answer Checking Prompt: Direct Evaluation

I have an input text and list of texts that presumably have the similar meanings. Your task is to decide how many of those text from the list have similar meaning. The input text is the following **input_text**, and the list of texts is the following: **answer_options**. Your output should only be a single floating point value of the resulted ratio like 0.3 or 0.6 and nothing else.

Result: Direct Evaluation

0.5

Although, this variant seemed to give the most concrete answers, its performance is very bad. It was wrong in nearly in every case, more specifically it was very strict/intolerant with difference in number, person and tense of the predicate. For this reason, we have further improved the prompt to eliminate this vulnerability. In addition, we have made changes to the response generation structure to a format where every single option is accepted individually, expressed by a True or False. The prompt is as follows:

Answer Checking Prompt: Individual Evaluation

I have an input text and list of texts that presumably have the similar meanings. Your task is to decide how many of those text from the list have similar meaning or have matching keywords. The acceptance criteria is to cover the same topic. We don't require an exact match. The input text is the following: **input_text** and the list of texts is the following: **answer_options**. Your output should only be a list of id-s and bools denoting if the was corresponding to the input text. Return a well formed .json only.

Result: Individual Evaluation

```
{ "results": { "0": true, "1": true, "2": true, "3": false, "4": false, "5": false, "6": false, "7": false, "8": false, "9": false } }
```

This iteration was way more stable and consistent but it got too accepting. On average it accepted 10 to 15% more of the answer option that it should have. Therefore a next variant was tried which incorporated the **One-Shot** method [31]. We have selected a ex-

ample from the test set, manually evaluated and then inserted into the prompt. Both the input and output parts were used. The prompt is as follows:

Answer Checking Prompt: Individual Evaluation with One-Shot Learning

I have an input text and list of texts that presumably have the similar meanings. Your task is to decide how many of those text from the list have similar meaning or have matching keywords. The acceptance criteria is to cover the same topic. We don't require an exact match. An example for this process is the following: input text is 'Furnace 1 3 sliding cylinder is tightened . ', list of possible alternatives: { '0': 'Furnace 1 3 sliding cylinder is tightened . ', '1': 'Furnace 1 3 sliding cylinder error . ', '2': 'Furnace 1 3 sliding cylinder error . ', '3': 'Furnace 1 sliding cylinder did not go to end . ', '4': 'Furnace 1 3 with a cylinder did not go in . ', '5': 'Furnace 1 , 3-inch cylinder . ', '6': 'Furnace 1 once again stopped 3 sliding cylinder with fault . ', '7': 'Furnace 2 1 sliding cylinder outside does not indicate . ', '8': 'Furnace 1 3 sliding cylinder inside the end stop conflicting down . ', '9': 'Furnace 1 2 with a sliding cylinder defect stops . ' } and the correct output is { "0": True, "1": True, "2": True, "3": False, "4": False, "5": False, "6": True, "7": False, "8": True, "9": False }. The real input text is the following: **input_text**, and the list of texts is the following: **answer_options** Your output should only be a list of id-s and bools denoting if the was corresponding to the input text. Return a well formed .json only

Result: Individual Evaluation with One-Shot Learning

```
{ "0": true, "1": true, "2": true, "3": false, "4": false, "5": false, "6": false, "7": false,
  "8": false, "9": false }
```

The first difference was a slight alteration in the output, because this time the "results" keyword has disappeared. Most likely this can be derived from specifically formatted output in the input prompt. But more importantly, this prompt performed very well on the test set and it was within 5% of giving the same evaluations as we gave to the answer sets. In conclusion, the third evolution of the answer verification prompt was chosen.

Chapter 9

Final Results

9.1 Numerical Results (Vectorial Searching)

In this section, we present the results of our numerical investigation, which aimed to analyze the performance of the various clustering methods employed in our study. To illustrate our findings, we utilized two key visualizations: a scatter plot depicting the average values across all cases and a distribution plot that highlights the spread of these averages. The scatter plot provides a clear representation of the central tendencies observed in the data, allowing for a straightforward comparison of the clustering outcomes. Meanwhile, the distribution plot offers insights into the variability and overall distribution of the average values, revealing patterns that may not be immediately apparent in the scatter plot alone. Together, these visualizations serve to enhance our understanding of the results and facilitate a comprehensive analysis of the effectiveness of the different algorithms.

The following two plots will present the results of the Vectorial Search in case of the base text and the minimally and maximally augmented text:



Figure 9.1: Vector Search Numerical Attributes Regression Analysis

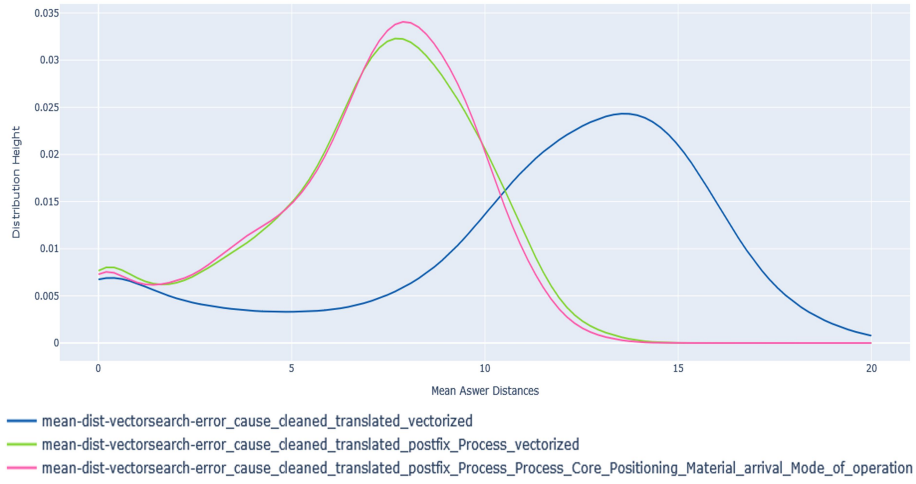


Figure 9.2: Vector Search Numerical Attributes Distribution

Based on these figures, some simple but important conclusions could be made:

1. Based on the regression lines of figure 9.1, across all answer search cases we could find options with better numerical attributes through the querying of augmented text embeddings.
2. Even though, the augmented embeddings result better findings, the minimal and a maximal augmentation do not really diverge from each other, but the maximal augmentations performs slightly better as show on figure 9.2(although, it is withing the error limit),
3. The general characteristics of these findings are also true for all the other answer selection methods. For all three clustering algorithms the base text embeddings give the results with the worst numerical possibilities.

Next, let's see how these answer options are evaluated by the model. Just like in the case of the numerical attribute calculation, first we have scatter plotted the acceptance ratio of the test cases, calculated the regression lines, and for second, we have calculated the distribution curves. Now let's see the results:



Figure 9.3: Vector Search GPT Verification Regression Analysis

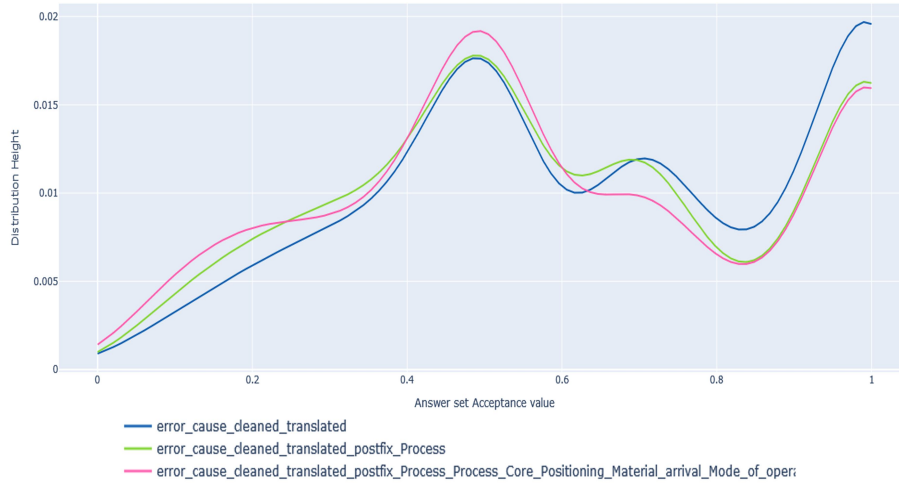


Figure 9.4: Vector Search GPT Verification Distribution

Now let's see, what conclusions were made:

1. Contrary to the expectation of the numeric analysis, based both on figure 9.3 and 9.4 we can say, that the non augmented embeddings lead to better/more similar answer to the inputs. There is a larger difference, especially in the region of 100% and close to it.
2. Again, contrary to what the numerical analysis predicted, this time the minimally augmented embeddings were the basis for the better answer selection.

9.2 Answer Verification Metrics

Given the fact that the most important metrics could be summed up in simple numbers, for the remaining the algorithms only the answer verification results will be included expressed as the average of the 1084 cases.

Table 9.1: Vector Search Accuracy

Entrie / Accuracy	Recall	Pass 10
Base	55.9%	100.0%
Aug. 1	52.4%	100.0%
Aug. 5	52.1%	100.0%

Table 9.2: Hierarchical Clustering Accuracy

Entrie / Accuracy	Recall	Pass 10
Base	47.7%	100.0%
Aug. 1	46.9%	100.0%
Aug. 5	45.5%	100.0%

Table 9.3: KMeans Clustering
Accuracy

Entrie / Accuracy	Recall	Pass 10
Base, k=35	48.3%	97.2%
Base, k=9	51.5%	92.4%
Base, k=40	51.8%	90.0%
Base, k=49	51.9%	90.9%
Base, k=28	54.5%	91.5%
Aug. 1, k=13	50.8%	94.0%
Aug. 1, k=17	55.7%	93.1%
Aug. 1, k=27	56.3%	95.2%
Aug. 1, k=32	52.1%	92.3%
Aug. 1, k=50	53.3%	98.2%
Aug. 5, k=32	52.2%	91.5%
Aug. 5, k=10	58.9%	92.3%
Aug. 5, k=21	50.6%	99.9%
Aug. 5, k=28	60.1%	92.2%
Aug. 5, k=35	56.8%	90.4%

Table 9.4: DBSCAN Clustering
Accuracy

Entry / Accuracy	Recall	Pass 10
Base, 0.620 - 1	75.1%	88.2%
Base, 0.486 - 2	75.1%	88.2%
Base, 0.425 - 3	52.4%	91.9%
Base, 0.391 - 4	52.1%	98.6%
Base, 18.60 - 25	52.7%	96.7%
Aug. 1, 0.921 - 1	73.2%	84.2%
Aug. 1, 10.52 - 13	56.3%	85.0%
Aug. 1, 10.53 - 16	57.1%	84.1%
Aug. 1, 10.58 - 18	56.9%	84.1%
Aug. 1, 7.10 - 1	66.6%	95.8%
Aug. 5, 0.986 - 1	73.2%	84.2%
Aug. 5, 10.120 - 25	57.2%	89.8%
Aug. 5, 9.85 - 12	57.4%	89.6%
Aug. 5, 9.857 - 24	58.0%	86.2%
Aug. 5, 9.793 - 9	57.2%	89.9%

Based on these numbers, we can only make a few clear conclusions.

1. The Vector Search mechanism and Hierarchical Clustering have the absolute strength of finding at least one correct answer and the remaining struggles with this.

2. On the other hand, there were cases for both the KMeans and the DBSCAN algorithms where they performed better at the average score, especially the DBSCAN.
3. Hierarchical Clustering clearly underperformed the Vector Search algorithm.
4. In case of Hierarchical Clustering and the Vector Search is clear that the simple embeddings result worse answer selections, but in case of the other two, this can not be stated.

Chapter 10

Summary of the Thesis

10.1 General results

In this study, we aimed to develop a robust system for grouping text entries based on their semantic similarities. The ability to effectively categorize texts is essential for various applications, including information retrieval, content recommendation, and sentiment analysis. To achieve this goal, we embedded the text entries into a numerical format that could be processed by machine learning algorithms. By employing vector search techniques and three distinct clustering algorithms, we sought to identify the most effective method for finding similar meanings in new input texts.

Fortunately, all proposed investigation ideas were executable and amenable to experimentation. Each algorithm was implemented successfully, allowing us to explore their respective strengths and weaknesses in the context of our text embedding system. The experiments provided valuable insights into the performance of the different methods, enabling us to assess their effectiveness in grouping semantically similar texts. This comprehensive approach ensured that we could gather a wealth of data to inform our analysis and conclusions.

Despite the thorough investigation and successful execution of our experiments, we encountered a significant challenge: there was no clear winner among the clustering algorithms. The final scores obtained from the evaluations were notably noisy, making it difficult to draw definitive conclusions about the superiority of any single method. This variability in results highlighted the complexities inherent in semantic similarity assessments and underscored the need for further refinement and exploration of the algorithms. As a result, while we gained valuable insights, the quest for the most effective clustering

approach remains an ongoing endeavor.

10.2 Evaluation of the initial Hypothesis

Concluding the hypothesis set at the beginning of the research, we have investigated whether they could be accepted or they have to be rejected.

1. Augmentation:

HA0: Attaching contextual information on text level to the raw entries will diverge those that has high similarity at base form. - **Accepted**, since lower cluster number was enough for a higher silhouette measure clustering (see figure 7.6).

HA1: Attaching contextual information by vectorial operation to the raw entries will diverge those that has high similarity at base form. - **Rejected**, since it changes the embeddings completely chaotically compared to the original (see figure 6.3).

HA2: Too much contextual information will have noise effect on the entries, meaning, that the resulted groups will resemble more the contextual similarities between the added augmentations. - **Accepted**, since in 3 out the 4 methods, the maximal augmentation perform worse than the minimal, and even in case of the fourth one (DBSCAN) it is undecided.

HA3: Text level augmentation (Early fusion) will perform better than the separate vectorization and vector level operation augmentation (Late fusion) of the additional data. - **Accepted**, since it changes the embeddings completely chaotically compared to the original (see figure 6.3).

2. Entrie grouping and Answer selection:

HEg2: Hierarchical clustering will have a better grouping effect of similar instances than direct vectorial searching of closest neighbors. - **Rejected**, since table 9.2 proves the opposite.

HEg1: KMeans clustering will have a better grouping effect of similar instances than direct vectorial searching of closest neighbors. - **Plausible**, since not all measures are better, but some are (see table 9.3).

HEg2: DBSCAN algorithm will have a better grouping effect of similar instances than direct vectorial searching of closest neighbors. - **Plausible, since not all measures are better, but some are** (see table 9.4).

HEg3: KMeans clustering will perform the best out of the investigated clustering algorithms. - **Undecided, since Hierarchical clustering tend to be the worst, but between KMeans and DBSCAN it cannot be decided** (see tables 9.3 and 9.4).

3. Answer Verification:

HAV0: Within the 10 best matches of the answer possibilities by numerical attributes will contain at least one right answer. - **Rejected, since not all the algorithms had a 100% absolute accuracy score** (see tables 9.3 and 9.4).

HAV2: Within the 10 best matches more than half will be an acceptable answer option. - **Rejected, since not all the algorithms had a more than 50% Recall score** (see table 9.2).

HAV3: A simple output prompt (like returning just a number) for verifying the answer set, will be able to provide trustworthy results. - **Rejected, since the One shot method was required for a consistent analysis.**

10.3 Future developments

As we looked toward future developments in our text grouping system, several avenues for enhancement emerged. These potential improvements encompassed various parameterization strategies and utilization methods for the algorithms we investigated, as well as opportunities for data augmentation and the exploration of alternative techniques. By refining our approaches, we aim to increase the accuracy and efficiency of clustering similar text entries, ultimately leading to a more robust system capable of handling diverse datasets and complex semantic relationships.

In terms of augmentation, we recognized that enhancing the input data could significantly impact the effectiveness of our algorithms. One approach involved incorporating additional contextual information, such as metadata or user-generated tags, to enrich the text embeddings. This supplementary data could provide valuable insights into the semantic relationships among texts, leading to more accurate groupings. Additionally, experimenting with different embedding techniques, such as alternative concatenation rules,

non transformer-based models or domain-specific embeddings, could further improve the representation of textual information and enhance the clustering results.

In the case of the KMeans algorithm, one significant area for future exploration involves the optimization of the k parameter. While we initially relied on traditional methods such as the elbow method, we recognized that incorporating advanced techniques like the Davies-Bouldin index could provide deeper insights into the optimal number of clusters. Additionally, implementing adaptive clustering mechanisms that adjust the number of clusters based on the data's inherent characteristics could enhance the algorithm's responsiveness to varying input. Promising directions also include refining the centroid initialization process or super cluster division. These changes could lead to more effective clustering and reduce the risk of converging on suboptimal solutions.

For KMeans, another potential development involved experimenting with different distance metrics. While we primarily utilized Euclidean distance, exploring alternatives such as cosine similarity or Manhattan distance could yield different clustering outcomes, particularly in high-dimensional spaces. This flexibility in distance measurement might allow for a more nuanced understanding of the relationships among text embeddings. Furthermore, integrating ensemble methods that combine the results of multiple clustering runs could enhance the robustness of the final groupings, providing a more comprehensive view of the data.

When considering the DBSCAN algorithm, future work could focus on optimizing the ϵ and min_samples parameters through automated tuning techniques. Implementing methods such as grid search or genetic algorithms could facilitate the identification of ideal parameter settings without extensive manual intervention. Additionally, exploring the use of alternative distance metrics beyond Euclidean could improve the algorithm's ability to capture complex relationships within the data. This adaptability would allow DBSCAN to better accommodate the diverse shapes and densities of clusters present in text embeddings.

Another avenue for DBSCAN's development involved the integration of hybrid approaches that combine its strengths with other clustering methods. For instance, using a two-step process where initial clusters are identified using DBSCAN and then refined through KMeans could yield more accurate results. This combination would leverage the density-based capabilities of DBSCAN while benefiting from the efficiency of KMeans in handling well-defined clusters. Such hybrid models could provide a more comprehensive analysis of text embeddings and enhance overall clustering performance.

Lastly, we consider the possibility of integrating feedback mechanisms into the system to refine the clustering process continually. By allowing users to provide input on the quality of the groupings, we could develop a more adaptive system that learns from user interactions. This iterative approach could lead to ongoing improvements in the algorithms' performance, as they would be able to adjust their parameters and methods based on real-world usage and user preferences. Such developments would not only enhance the accuracy of the text grouping system but also contribute to a more user-centered design, ultimately improving the overall user experience.

In addition to the mentioned methods, we also contemplate the integration of advanced machine learning techniques, such as deep learning models, to enhance the clustering process. Utilization of neural networks for feature extraction could provide richer representations of text data, potentially leading to improved clustering outcomes. By leveraging the power of deep learning, we could capture more complex semantic relationships that traditional methods might overlook.

Furthermore, exploring the use of graph-based clustering techniques could offer another layer of insight into the relationships among text entries. By representing texts as nodes in a graph and utilizing algorithms such as spectral clustering or community detection, we could uncover hidden structures within the data. This approach could complement the existing clustering methods and provide a more comprehensive understanding of the semantic landscape.

Finally, we recognize the potential benefits of incorporating unsupervised learning techniques alongside our clustering algorithms. By utilizing methods such as topic modeling or latent semantic analysis, we could gain additional insights into the underlying themes present in the text data. This complementary analysis could enhance our understanding of the relationships among texts and inform the clustering process, ultimately leading to more meaningful groupings.

Bibliography

- [1] *Robert Bosch Elektronika Kft. - Finance*. URL: <https://webshop.opten.hu/robert-bosch-elektronika-kft-e11182910.html>.
- [2] Berry Matijssen. *The Cost of Manufacturing a Car*. 2023. URL: <https://stocoto.com/the-cost-of-producing-a-car/>.
- [3] *Hatvan – Robert Bosch Elektronika Kft*. URL: <https://www.bosch.hu/en/our-company/bosch-in-hungary/hatvan/>.
- [4] Mahesh S Raisinghani et al. “Six Sigma: concepts, tools, and applications”. In: *Industrial management & Data systems* 105.4 (2005), pp. 491–505.
- [5] Geoff Tennant. *Six Sigma: SPC and TQM in manufacturing and services*. Routledge, 2017.
- [6] Stefan H Steiner, R Jock MacKay, and John S Ramberg. “An overview of the Shainin System™ for quality improvement”. In: *Quality Engineering* 20.1 (2007), pp. 6–19.
- [7] Marcelo Luiz Eugênio. *Contribution for Statistical Engineering through a case study applying Shainin Red X strategies method*. Tech. rep. SAE Technical Paper, 2006.
- [8] Laura Labanowski. “Improving overall fabricator performance using the continuous improvement methodology”. In: *1997 IEEE/SEMI Advanced Semiconductor Manufacturing Conference and Workshop ASMC 97 Proceedings*. IEEE. 1997, pp. 405–409.
- [9] Lakhan Patidar, Vimlesh Kumar Soni, and Pradeep Kumar Soni. “Continuous improvement philosophy for manufacturing productivity: critical review”. In: *Trends in Industrial and Mechanical Engineering* 30 (2016), pp. 34–45.

- [10] Liliana Luca. “A new model of Ishikawa diagram for quality assessment”. In: *IOP Conference Series: Materials Science and Engineering* 161 (Nov. 2016), p. 012099. DOI: 10.1088/1757-899X/161/1/012099.
- [11] C Botezatu et al. “Use of the Ishikawa diagram in the investigation of some industrial processes”. In: *IOP Conference Series: Materials Science and Engineering* 682 (Nov. 2019), p. 012012. DOI: 10.1088/1757-899X/682/1/012012.
- [12] Carolyn H Hopper. “Practicing college learning strategies”. In: (*No Title*) (2013).
- [13] AMM Sharif Ullah. “Fundamental issues of concept mapping relevant to discipline-based education: A perspective of manufacturing engineering”. In: *Education sciences* 9.3 (2019), p. 228.
- [14] Zijian Győző Yang. “BARTerezzünk! - Messze, messze, messze a világtól, - BART kísérleti modellek magyar nyelvre”. In: *XVIII. Magyar Számítógépes Nyelvészeti Konferencia*. Szeged, Magyarország: Szegedi Tudományegyetem, Informatikai Intézet, 2022, pp. 15–29.
- [15] Jacob Devlin et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2019. arXiv: 1810.04805 [cs.CL]. URL: <https://arxiv.org/abs/1810.04805>.
- [16] Xin Zhang et al. “mGTE: Generalized Long-Context Text Representation and Reranking Models for Multilingual Text Retrieval”. In: *arXiv preprint arXiv:2407.19669* (2024).
- [17] Zehan Li et al. “Towards general text embeddings with multi-stage contrastive learning”. In: *arXiv preprint arXiv:2308.03281* (2023).
- [18] Hongjin Su et al. *One Embedder, Any Task: Instruction-Finetuned Text Embeddings*. 2023. arXiv: 2212.09741 [cs.CL]. URL: <https://arxiv.org/abs/2212.09741>.
- [19] Praveen Badimala et al. “A study of various text augmentation techniques for relation classification in free text”. In: *8th International Conference on Pattern Recognition Applications and Methods*. SciTePress Digital Library. 2019, pp. 360–367.
- [20] Sultan Alshamrani. “Distance Matters: Euclidean Embedding Distances for Improved Language Model Generalization and Adaptability”. In: *IEEE Access* (2024).

- [21] Harald Steck, Chaitanya Ekanadham, and Nathan Kallus. “Is cosine-similarity of embeddings really about similarity?” In: *Companion Proceedings of the ACM Web Conference 2024*. 2024, pp. 887–890.
- [22] Nils Reimers and Iryna Gurevych. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. 2019. arXiv: 1908.10084 [cs.CL]. URL: <https://arxiv.org/abs/1908.10084>.
- [23] Junwei Zhou et al. “DeepSyslog: Deep Anomaly Detection on Syslog Using Sentence Embedding and Metadata”. In: *IEEE Transactions on Information Forensics and Security* 17 (2022), pp. 3051–3061. DOI: 10.1109/TIFS.2022.3201379.
- [24] T. Cover and P. Hart. “Nearest neighbor pattern classification”. In: *IEEE Transactions on Information Theory* 13.1 (1967), pp. 21–27. DOI: 10.1109/TIT.1967.1053964.
- [25] Roger Alan Stein, Patricia A Jaques, and Joao Francisco Valiati. “An analysis of hierarchical text classification using word embeddings”. In: *Information Sciences* 471 (2019), pp. 216–232.
- [26] Jayasree Ravi and Sushil Kulkarni. “Text embedding techniques for efficient clustering of twitter data”. In: *Evolutionary Intelligence* 16.5 (2023), pp. 1667–1677.
- [27] Shapol M Mohammed, Karwan Jacksi, and Subhi RM Zeebaree. “Glove word embedding and DBSCAN algorithms for semantic document clustering”. In: *2020 international conference on advanced science and engineering (ICOASE)*. IEEE. 2020, pp. 1–6.
- [28] Gloriana Monko and Masaomi Kimura. “Optimized DBSCAN parameter selection: Stratified sampling for epsilon and gridsearch for minimum samples”. In: *Computer Science & Information Technology (CS & IT)* (2023), pp. 43–61.
- [29] Abhinay Gupta, Praveen Kumar Tyagi, and Ebtasam Ahmad Siddiqui. “Question Pair Similarity Detection and Development of Question Answering Chatbot”. In: *2025 IEEE International Students’ Conference on Electrical, Electronics and Computer Science (SCEECS)*. 2025, pp. 1–6. DOI: 10.1109/SCEECS64059.2025.10940119.
- [30] Vijayan Naveen Edapurath. “Enhancing Chatbot Response Relevance through Semantic Similarity Measures”. In: (2022).

- [31] Hai Dang et al. *How to Prompt? Opportunities and Challenges of Zero- and Few-Shot Learning for Human-AI Interaction in Creative Applications of Generative Models*. 2022. arXiv: 2209.01390 [cs.HC]. URL: <https://arxiv.org/abs/2209.01390>.

List of Figures

1.1	Current Problem Solving Workflow	5
1.2	Improved workflow, green boxes representing the newly integrated steps	6
3.1	Six Sigma	12
3.2	Shainin Red X	13
3.3	Lean-CIP	14
3.4	Ishikawa-Fishbone diagram	15
3.5	The Mind Map	16
3.6	Framework Evolution	16
3.7	Complication Understanding Compass	22
4.1	Distribution of production_line and workplace and their combination	26
4.2	Examples and Distribution for "error_code" feature	27
4.3	Character/Token length comparison for Hungarian and English texts	27
4.4	Repair time distribution of the machine stops	28
5.1	Workflow of the developed subsystem	36
6.1	Augmented Embedding distances	39
6.2	Augmented Embedding distances	40
6.3	Augmented Embedding distances	41
7.1	Visible differences between the same 1000 text embeddings at different augmentation level	44
7.2	Single and Average linkage	46
7.3	Complete and Ward linkage	46
7.4	Silhouette score for raw text embeddings	48
7.5	KMeans Clustering Inertia Score	50
7.6	KMeans Clustering Inertia Score Change	51
7.7	KMeans Clustering Silhouette Score	52

LIST OF FIGURES

7.8	KMeans Clustering Silhouette Score Change	52
7.9	Step 1: Neighbor Distance Spectrum Base Texts	55
7.10	Step 2: Neighbor Distance Distribution Base Texts	56
7.11	Step 3: Hyperparameter Field Base Texts	56
7.12	Single and Average linkage	57
9.1	Vector Search Numerical Attributes Regression Analysis	65
9.2	Vector Search Numerical Attributes Distribution	65
9.3	Vector Search GPT Verification Regression Analysis	66
9.4	Vector Search GPT Verification Distribution	66

List of Tables

5.1	Databricks Computational Capacity	31
5.2	Character Encoding Correction	34
5.3	Spell Checking word correction	34
5.5	Translation post-processing cases	34
5.4	Character Sequences Alterations	35
7.1	Hierarchical Silhouette Score	48
7.2	KMeans Change Score	53
7.3	DBSCAN Silhouette Score	58
9.1	Vector Search Accuracy	67
9.2	Hierarchical Clustering Accuracy	67
9.3	KMeans Clustering Accuracy	67
9.4	DBSCAN Clustering Accuracy	67